TABLE OF CONTENTS

Abstract 3

Preface to Websites and Web Applications 4-5

- What is a Website ?
- Key Technologies Used in Websites
- Other components
- Types of Websites
- How These Are Connected ?

1. Introduction 6-11

- Overview of the Farmer-to-Consumer Marketplace
- Key Features and Purpose
- Objectives
- Workflow
- Project Structure

2. System Architecture 12-14

- Introduction
- Components & Their Roles
- Architecture Flow
- Working Architecture through Graphics

3. Conceptual Diagram 15-19

- Frontend (HTML,CSS,JavaScipt,Bootstrap)
- Flask Backend(Python)
- MongoDB Atlas (Cloud Database)
- Cashfree Payment Gateway API

4. Modules Explanation 20-54

- App.py (Main Application)
- Customer.py (Manages Customer Orders)
- Farmer.py (Manages Farmer Dashboard)
- Payment_gateway.py (Handles Payments via Cashfree API)

- Db.py (Handles MongoDB Operations)
- Config.py (Stores Configuration Settings)
- common.py(the common routes like home,logoutand profile)

5. Database Design 55-59
- Database Schema Overview
- Farmers Collection
- Customers Collection
- Vegetables Collection
- Carts Collection
- Records Collection
- Data Flow Explanation
- Summary

6. API Documentation 60-62
- Overview
- Authentication & User Management
- Vegetable & Order Management
- Payment Gateway (Cashfree)
- Order & Delivery Management
- Error Handling
- Security Measures

7. Testing and Results 63-64
- Testing Setup
- Test Cases and Observations
- Observations

8. Application Demonstration 65-73

9. Future Enhancements 73-74
- Test Environment Setup
- Test Cases & Expected Outputs
- System Performance & Security Tests

10. Conclusion 75

Abstract

The traditional agricultural supply chain often involves multiple intermediaries, leading to reduced profits for farmers and increased costs for consumers. To address this issue, this project aims to develop a Farmer-to-Consumer Green Vegetable Marketplace, providing a direct platform where farmers can sell their produce to consumers without middlemen. This system enhances transparency, ensures fair pricing for farmers, and offers consumers fresher and more affordable vegetables.

The platform includes essential features such as farmer registration, product listing, real-time inventory management, and consumer authentication for seamless browsing and purchasing. The user-friendly interface is designed with HTML, CSS, and internal JavaScript, ensuring a smooth and accessible experience. The backend is built using the Flask framework, offering a lightweight yet efficient structure to handle server-side operations. MongoDB Atlas is integrated as the database to manage user data, product listings, and transaction records effectively.

For secure and seamless transactions, the system incorporates Cashfree Payment Gateway, allowing consumers to make instant online payments. The elimination of middlemen not only benefits farmers by maximizing their earnings but also enhances consumer trust by providing direct access to farm-fresh produce. This digital marketplace fosters a sustainable agricultural ecosystem by empowering farmers with better control over pricing and sales while ensuring customers receive high-quality products at competitive prices.

This project represents a step forward in modernizing agricultural commerce by leveraging technology to create a fair, transparent, and efficient supply chain that directly connects farmers with consumers.

Preface to Websites and Web Applications

What is a Website?

A website is a collection of web pages accessible via the internet, typically identified by a domain name. It is designed to present information, services, or interactive experiences to users. Websites can range from simple static pages to complex, dynamic applications.

Key Technologies Used in Websites

1. **HTML (HyperText Markup Language)** – The backbone of web pages, defining the structure and content.
2. **CSS (Cascading Style Sheets)** – Used for styling and layout to enhance the visual appearance of a website.
3. **JavaScript** – Adds interactivity and dynamic behavior to web pages.
4. **Backend Technologies (Flask, Django, Node.js, etc.)** – Handle server-side logic, databases, and authentication.
5. **Databases (MongoDB, MySQL, PostgreSQL, etc.)** – Store and manage user data and transactions.
6. **APIs (Application Programming Interfaces)** – Enable communication between different services, such as integrating a payment gateway.

Other components

Here are some additional key aspects of websites:

1. **Web Hosting** – Websites need a hosting service to store files and make them accessible online. Hosting providers offer servers that ensure website availability and performance.

2. **Domain Name System (DNS)** – Translates human-readable domain names (e.g., www.example.com) into IP addresses, allowing users to access websites easily.

3. **Responsive Design** – Ensures that websites adapt seamlessly to different screen sizes and devices, improving user experience.

4. **SEO (Search Engine Optimization)** – Techniques used to improve a website's visibility in search engine results, helping attract more visitors.

5. **Security (SSL, Firewalls, Authentication)** – Protects user data and transactions by encrypting communications, preventing cyber threats, and ensuring secure access.

Types of Websites

Features	Static Website	Dynamic Website	Web Application
Definition	A simple website with fixed content that does not change unless manually updated.	A website that retrieves and displays data dynamically from a database.	A fully interactive web-based software designed for user input and complex operations.
Technology Used	HTML, CSS	HTML, CSS, JavaScript, PHP, Python, etc.	HTML, CSS, JavaScript, Flask, Django, React, etc.
Database	No database	Uses a database to store and retrieve content	Relies heavily on databases and APIs
User Interaction	Limited (Read-only)	Moderate (Users can interact with data)	High (Complex user interactions, authentication, etc.)
Examples	Blog, Portfolio	News Website, E-commerce Store	Social Media, Banking System

These Are How Connected?

A **static website** can be upgraded into a **dynamic website** by adding backend logic and database support. Further extending a dynamic website with advanced features like authentication, user roles, and real-time data processing transforms it into a **web application**.

For example, in the Farmer Consumer Marketplace:

- **Static Pages:** Home page, About Us
- **Dynamic Features:** Displaying available vegetables, order details, user authentication
- **Web Application Elements:** User authentication,Payment gateway, order tracking, real-time updates

1. Introduction

The Farmer Consumer Marketplace is a web application designed to connect farmers and customers, allowing customers to browse and order vegetables directly from farmers. It includes user authentication, a cart system, an order management system, and integration with the Cashfree Payment Gateway for secure transactions.

It's a Flask-based web application designed for secure online transactions between farmers and customers. It integrates Cashfree API for payment processing and uses MongoDB Atlas for order and transaction management.

Key Features and Purpose

The platform is designed to overcome common challenges in farmer-to-consumer transactions, such as **payment security, order tracking, and user accessibility**. To achieve this, it integrates the **Cashfree API** for secure online payments and utilizes **MongoDB Atlas** for efficient order and transaction management.

Objectives

1. Secure Payment Processing

- Ensures that all financial transactions are processed through a **secure and trusted payment gateway**.
- Uses encryption and authentication mechanisms to **prevent fraud and unauthorized access**.
- Provides a **smooth checkout experience**, reducing payment failures and improving customer trust.

2. Integration with Cashfree API

- The system integrates with **Cashfree**, a reliable and widely used payment gateway, for **transaction verification and processing**.
- Supports multiple payment methods, including **UPI, credit/debit cards, net banking, and wallets**.
- Automates **payment confirmation** and **status updates**, reducing manual intervention.

3. Order Management with MongoDB Atlas

- Stores and organizes **customer orders** in a **scalable, cloud-based MongoDB Atlas database**.
- Enables **quick retrieval of order details**, ensuring **efficient tracking** of pending, completed, or canceled orders.
- Allows **farmers to manage their sales records easily**, improving their business operations.

4. Transaction Tracking and Payment Status Updates

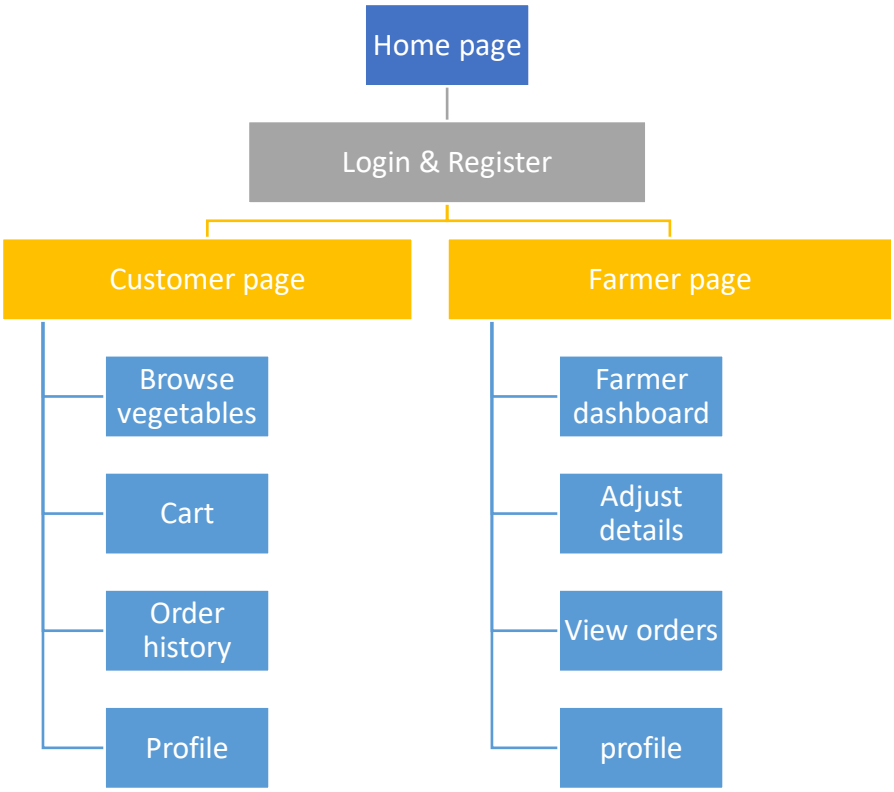
- Maintains a **detailed record of all transactions**, including:
    - Order details (product, quantity, price)
    - Payment status (successful, pending, failed)

    - Customer information
    - Timestamp of the transaction
- Provides a **dashboard or order history section** for users to track their past purchases.
- Ensures that **disputes or payment failures** can be quickly identified and resolved.

5. User-Friendly Interface

- Designed with a **simple, intuitive web-based UI**, making it easy for both farmers and customers to navigate.
- Allows users to **enter order details and complete transactions effortlessly**.
- Optimized for **mobile and desktop users**, ensuring accessibility across device

Workflow:



This diagram represents the structure of a web application with two main user roles: **Customer** and **Farmer**.

- **Home Page**: The entry point of the application.
- **Login & Register**: Users must log in or sign up to access their respective pages.

Customer Page

- **Browse Vegetables**: View available vegetables for purchase.
- **Cart**: Store selected items before checkout.

- **Order History**: Track past purchases.
- **Profile**: Manage customer's personal details.

Farmer Page

- **Farmer Dashboard**: Overview of sales and activity.
- **Adjust Details**: Update product or personal information.
- **View Orders**: Check customer orders.
- **Update Status**: Update the status of ordered item like shipped ,cancelled or delivered
- **Profile**: Manage farmer's personal details.

This structure ensures smooth interaction between customers and farmers in an online vegetable marketplace.

Project Structure

The project is built using Flask (Python) as the backend framework and MongoDB Atlas as the database. The application follows a Blueprint-based structure, making it modular and easy to manage.

```
/farm_fresh_project
|— templates/          # HTML Templates for frontend
        |—adjust_price.html
        |—browse_vegetables.html
        |—checkout.html
        |—farmer_dashboard.html
        |—gateway.html
        |—home.html
        |—order_history.html
        |—payment_status.html
        |—profile.html
        |—view_cart.html
        |—view_orders.html
|— static/             # Static files (CSS, JS, images)
        |—css/
                |—home.css
        |—icons/
```

```
            |—dehaat.png
            |—farmizen.png
            |—ninjacart.png
        |—uploads/   #vegetable photos
|— app.py              # Main entry point for the Flask application
|— config.py           # Application configuration (MongoDB, API keys, etc.)
|— db.py               # Database connections and collections
|— common.py           # Common helper functions
|— payment_gateway.py  # Cash free Payment Gateway Integration
|— customer.py         # Customer-related routes (browse, cart, order history)
|— farmer.py           # Farmer-related routes (manage vegetables, orders)
|— requirements.txt    # List of dependencies
```

Entry point – app.py

This module is the main part in which initializes flask(backend) and all the below modules are imported here and routes related to customer and farmer part are registered here.

Configuration – config.py

This module stores environment variables and configuration settings.

Key Features:

1.Flask Configuration

- SECRET_KEY: Security key for session management.
- UPLOAD_FOLDER: Directory for storing uploaded images.
- ALLOWED_EXTENSIONS: Allowed file formats for uploads.
- DEBUG: Enables or disables debugging.

2.MongoDB Configuration

- MONGO_URI: Connection string for MongoDB Atlas.
- The database name is farm_fresh.

3.Cashfree API Configuration

- CASHFREE_CLIENT_ID & CASHFREE_CLIENT_SECRET: Credentials for payment processing.
- CASHFREE_ENVIRONMENT: Sandbox or production mode.

Database Connection – db.py

This file establishes a MongoDB connection using pymongo.

Collections Used:

farmers _collection → Stores farmer details.

customers_collection → Stores customer details.

vegetables_collection → Stores vegetable inventory.

carts_collection → Stores items added to the cart.

history_collection → Stores order history.

records_collection → Stores payment records.

Customer Module – customer.py

This Blueprint manages customer operations, including browsing vegetables, adding to the cart, and ordering.

1. Browse Vegetables (/browse_vegetables)

   Displays available vegetables from vegetables_collection.

   Users can:

   - Add to Cart
   - Order Directly

2. View Cart (/view_cart)

   Retrieves the user's cart from carts_collection.

   Allows:

   - Updating quantity
   - Removing items
   - Proceeding to checkout

3. Order History (/order_history)

   Displays past orders from history_collection.

   Users can clear orders or view order details.

4. Checkout (/checkout)

   Retrieves cart data and stores the order in history_collection.

   Clears the cart and redirects to payment gateway.

5. Clear Orders

   Clear_selected_orders: Deletes selected orders from history.

   Clear_all_orders: Deletes the entire order history.

Farmer Module – farmer.py

This Blueprint handles farmer operations, including vegetable management and order tracking.

1. Farmer Dashboard (/farmer_dashboard/<username>)

   Displays the farmer's uploaded vegetables.

   Allows:

   - Adding new vegetables
   - Updating prices
   - Deleting vegetables

2. Order Management (/farmer_orders/<username>)

   Displays orders placed for the farmer's products.

   Allows status updates.

Payment Gateway Integration – payment_gateway.py

This module handles the Cashfree Payment Gateway integration.

Key Functionalities:

1. Initiate Payment (/payment)

   Generates an order with Cashfree API.

   Stores the payment details in records_collection.

2. Check Payment Status (/payment_status)

   Fetches transaction status from Cashfree API.

   Updates order status in history_collection.

Common functions – common.py

This module contains the code of home, register, login, profile and logout.

This is the place where authentication is done at login page and new users can register here. Their data will be safely stored at MongoDB Atlas.

It validates input ,role and phone number(10digits)

Session Management

Customers & Farmers must log in to access their respective dashboards.

Error Handling & Security

Flask flash messages notify users about actions.

Invalid actions redirect users with error messages.

MongoDB ObjectId validation prevents invalid queries.

Allowed File Extensions prevent unauthorized uploads.

2.System Architecture

The project follows a client-server architecture with a cloud-based backend, ensuring scalability, security, and efficient data management. The system is designed to facilitate seamless interactions between customers and farmers through an intuitive web interface. It leverages Flask for backend processing, MongoDB Atlas for data storage, and Cashfree for secure payment transactions. The system also includes security mechanisms, caching strategies, API integrations, and load-balancing techniques to enhance performance.

2.1. Components & Their Roles

2.11. User Interface (UI)

- Developed using HTML, CSS, and JavaScript.
- Provides an intuitive interface for customers and farmers.
- Uses AJAX for dynamic content updates.
- Implements responsive design for accessibility across devices.

2.12 Flask Web Server

- Serves as the backend framework.
- Handles HTTP requests and routes them to appropriate modules.
- Implements RESTful APIs for modular service interactions.
- Manages authentication, session handling, and API security.

2.13 Database (MongoDB Atlas)

- Cloud-hosted NoSQL database for scalability and availability.
- Stores user data, product details, orders, and payment information.

2.14 Payment Gateway (Cashfree)

- Facilitates secure online transactions.
- Supports multiple payment modes (UPI, Net Banking, Cards, Wallets).
- Provides API-based transaction verification.

2.2. Architecture Flow

2.21 User Access

- Users access the web-based UI.
- Authentication and authorization are handled via JWT.
- Farmers list products, and customers place orders.

2.22 Request Handling

- Users interact with the UI, triggering HTTP requests to Flask.

- Flask routes requests to respective modules.
- Middleware is used for request validation and logging.

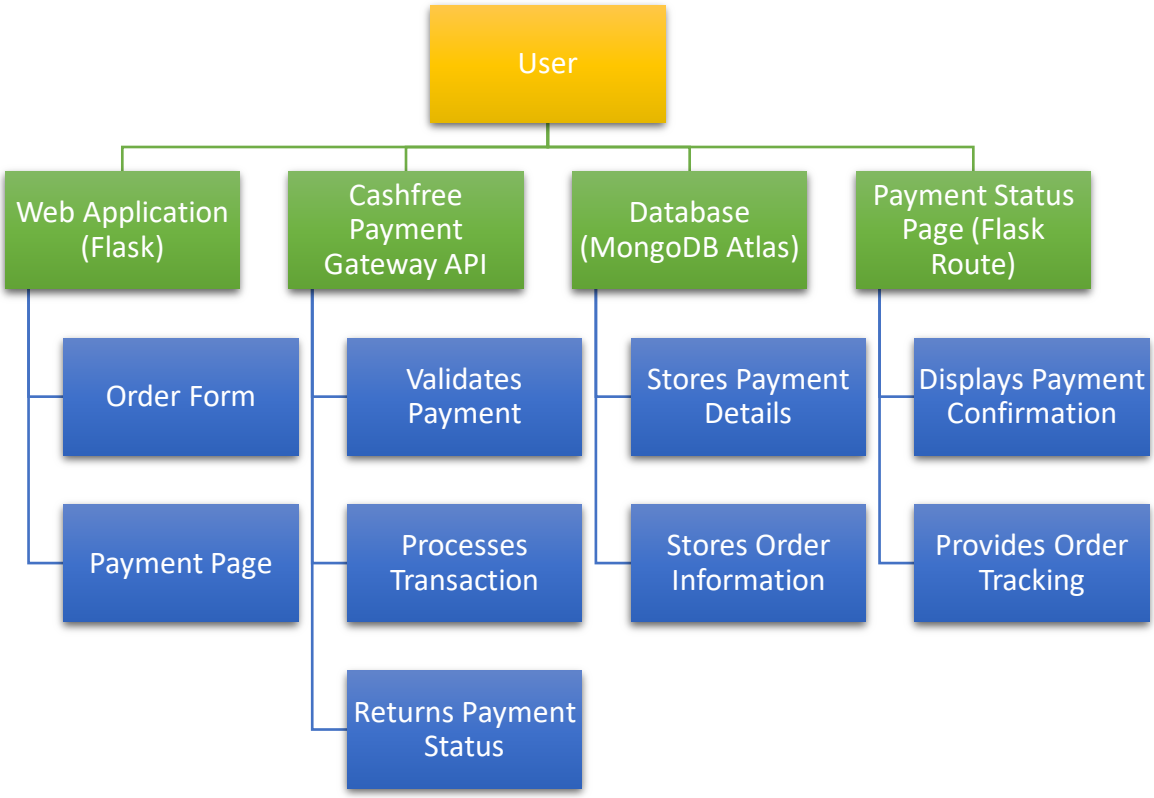
2.23 Database Interaction

- Flask communicates with MongoDB Atlas to fetch/update data.
- Optimized queries ensure fast response times.

2.24 Payment Processing

- Users initiate payments via Cashfree.
- Flask verifies transactions using Cashfree APIs.
- Payment status updates are stored in MongoDB.
- Failed transactions trigger retry mechanisms and user notifications.

System Architecture Diagram (Text Representation)



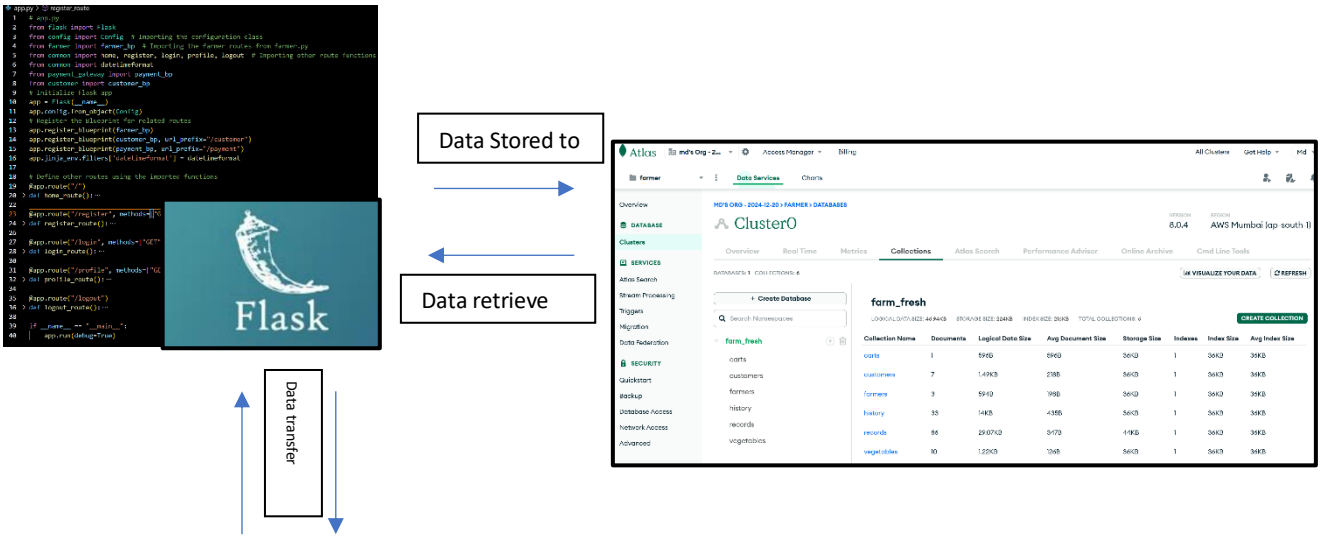
System Components Overview

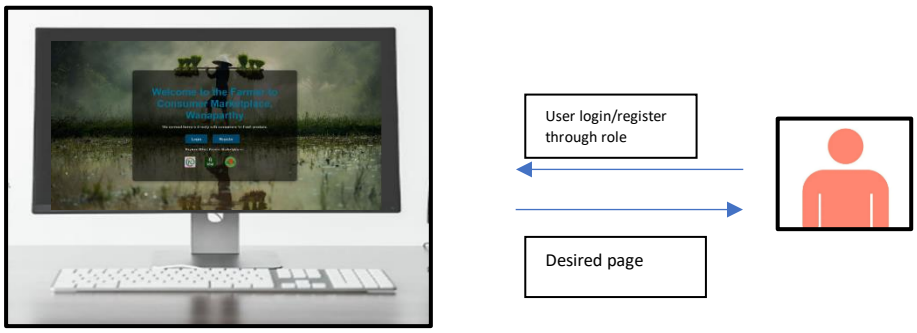
1. User Interface (HTML/CSS/JS)
   - The front-end of the platform where users interact.
   - Uses HTML for structure, CSS for styling, and JavaScript for dynamic functionality.
2. Flask Web Server

- A lightweight Python-based web framework handling backend logic.
- Manages user requests, processes payments, and interacts with the database.

3. MongoDB Atlas
   - A cloud-based NoSQL database storing order and transaction details.
   - Ensures scalable and efficient data management.
4. Cashfree Payment Gateway
   - A secure API for processing online payments.
   - Verifies transactions and updates payment statuses automatically.

Working Architecture though Graphics





The **Farmer Consumer Marketplace** follows a structured workflow for seamless transactions. Users **register or log in**, then access the platform to browse, place orders, and complete transactions.

The **Flask web server** processes user requests, manages business logic, and facilitates communication with **MongoDB Atlas**, a cloud-based NoSQL database storing user details, orders, and transactions. Data flows efficiently between the frontend, backend, and database, ensuring real-time updates.To enhance the system, integrating the **Cashfree Payment Gateway** in the architecture would illustrate the payment processing workflow. Overall, this design ensures a **secure, scalable, and user-friendly experience** for farmers and consumers

3.Conceptual Diagram

The webapp concept follows a modular approach, consisting of the following components.

3.1 Frontend (HTML, CSS, JavaScript, Bootstrap)

The frontend serves as the user interface for customers and farmers, providing interactive and user-friendly pages for various operations.

Key Functionalities:

Home Page (home.html):

Farmers and customers can register and login to the website & can also visit other websites

Profile(profile.html):

Farmer and customer can see their details & update the details.

Farmer Dashboard (farmer_dashboard.html):

Farmers can add, update, and delete vegetable listings.

Images of vegetables are previewed before upload.

Adjust details(adjust_price.html):

Farmers can adjust pricing ,update image and name dynamically.

Receive orders(view_orders.html):

Farmers can get details of customer & update status of order.

Customer page(browse_vegetables.html):

Customer can see the items ,place orders ,make payments.

Cart(view_cart.html):

All the items added to cart are visible here and update quantity and can remove from cart.

We can order multiple items in the cart by proceed to checkout button which redirect to payment gateway.

Order history(order_history.html):

Can see the ordered items, status & farmer details

Order & Payment Page (gateway.html):

Customers enter order details (Order ID, Name, Amount) and proceed to payment.

Payment status is displayed after processing.

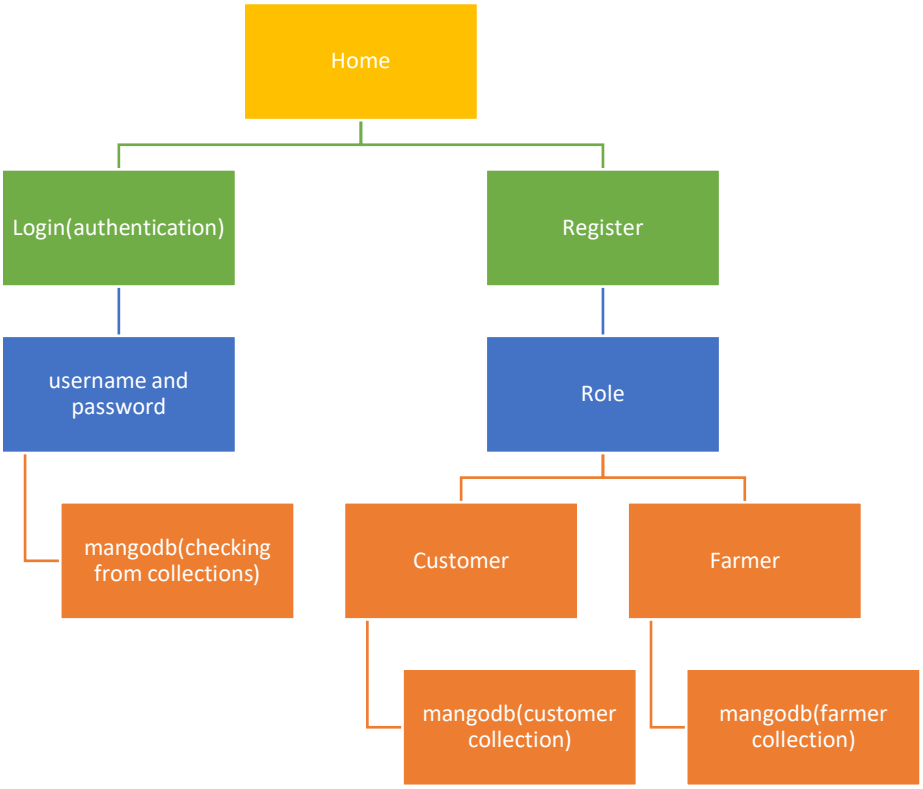
Checkout(checkout.html):

A button pay now will appear which directs to payment gateway

Payment status(payment_status.html):

It shows the order id ,status of the order whether its succeed or failed or pending and helps to go back to browsing vegetables.

Dataflow at frontend:



3.2 Flask Backend (Python)

The Flask backend serves as the core logic layer, handling requests, processing orders, and interacting with the database and payment gateway.

Key Functionalities:

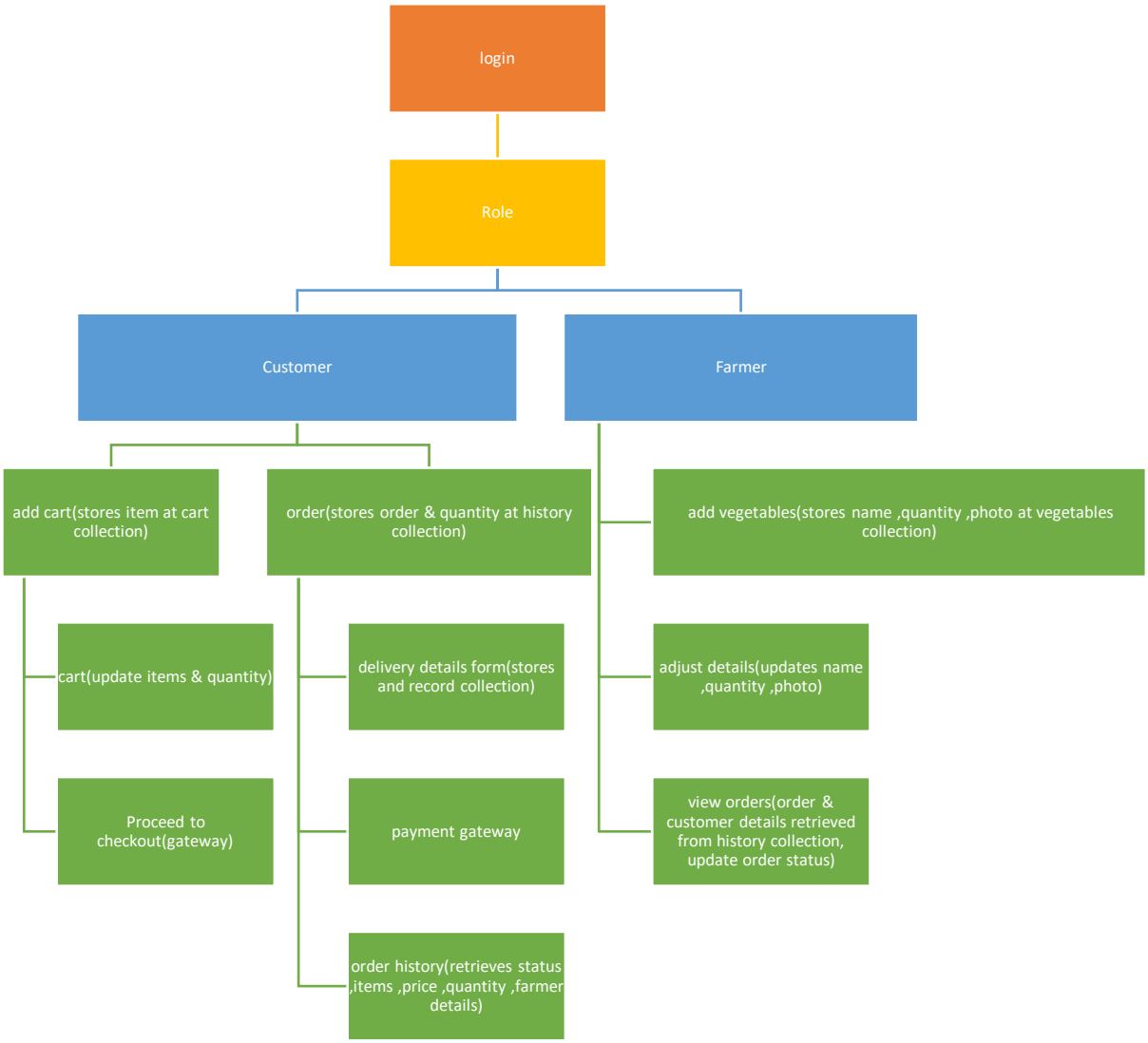
User Authentication (Role-based access: Farmer, Customer).

Order Management (Store, retrieve, and update order details).

Payment Processing via Cashfree API.

Database Management with MongoDB Atlas.

Data flow:



3.3MongoDB Atlas (Cloud Database)

MongoDB Atlas serves as the database layer, storing essential system information in collections.

Database Collections:

Users: Stores farmer and customer details.

Vegetables: Stores vegetable names, prices, and images.

Orders: Stores order details, including status and payment ID.

Payments: Stores transaction records from Cashfree API.

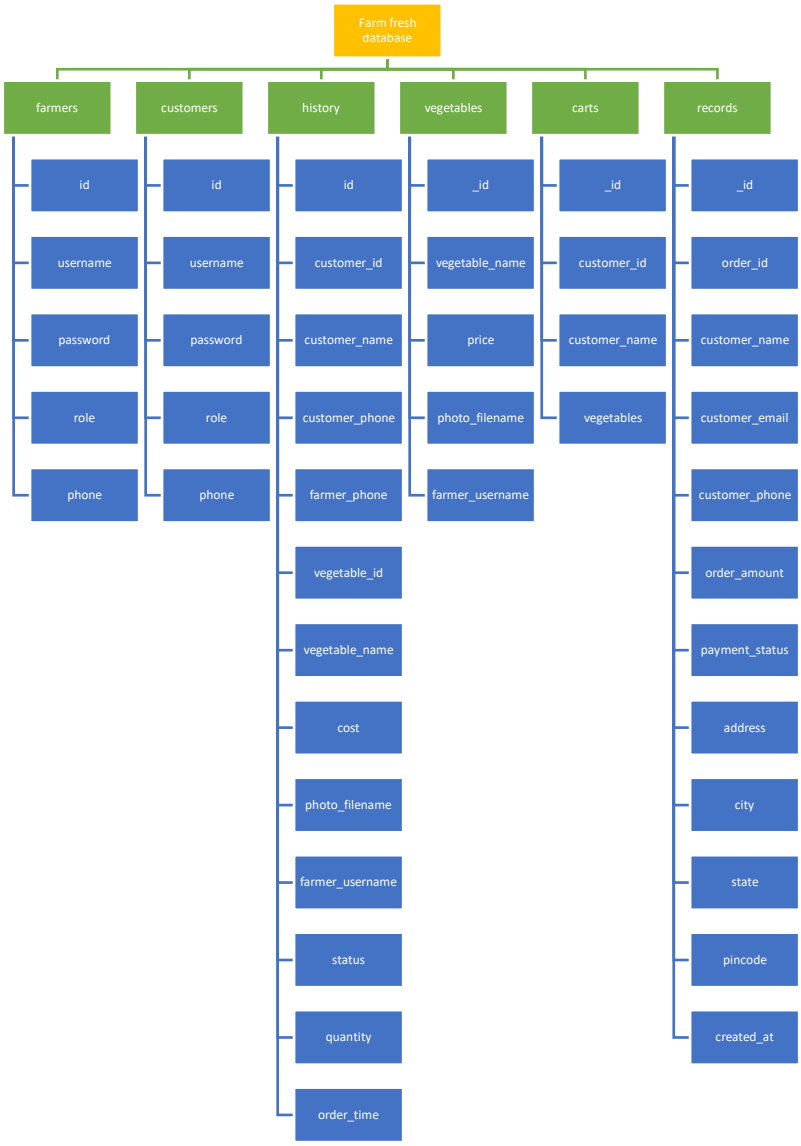
Key Features:

Cloud-based storage ensures scalability and security.

Indexing & Query Optimization for faster retrieval of vegetable listings and orders.

Schema-less flexibility for easy modifications.

Data flow:



3.4Cashfree Payment Gateway API

The system integrates Cashfree's secure API to handle online payments.

Key Functionalities:

Order Creation: Generates unique order IDs.

Payment Processing: Handles secure card and UPI payments.

Payment Verification: Retrieves real-time transaction status.

API Requests & Responses:

1. Creating an Order (POST Request to Cashfree API)

Sends order details (order_id, amount, customer_name, etc.).

Receives a payment link for transaction processing.

2. Fetching Payment Status (GET Request to Cashfree API)

Retrieves payment status (SUCCESS, PENDING, FAILED).

Updates the MongoDB payments collection accordingly.

Overall Workflow

1. Customer selects vegetables and proceeds to checkout.
2. Flask backend generates an order ID and requests payment via Cashfree API.
3. Customer completes the payment on Cashfree's hosted page.
4. Flask verifies the payment status via API and updates MongoDB Atlas.
5. Customer & farmer dashboards display updated order statuses

This modular architecture ensures seamless interaction between farmers and customers while enabling secure online transactions.

System Architecture of the Farmer Management System

4. Modules Explanation

Modules are divided into seven

1)app.py: Main entry point, handles routing.

2)customer.py: Manages customer orders.

3) farmer.py: Displays order and payment status.

4)payment_gateway.py: Integrates Cashfree API for payment processing.

5)db.py: Handles MongoDB operations.

6)config.py: Stores configuration settings.

7)common.py:Has the code of commonly used in both pages

1. App.py (Main Application)

Its the main entry point of your Flask application. It initializes the app, configures settings, registers Blueprints, and defines routes.

Breakdown of the Code

1. **Importing Required Modules**

```
from flask import Flask

from config import Config  # Importing the configuration class

from farmer import farmer_bp  # Importing the farmer routes from farmer.py

from common import home, register, login, profile, logout  # Importing other route functions

from common import datetimeformat

from payment_gateway import payment_bp

from customer import customer_bp
```

- **Flask:** The core framework to create the web app.
- **Config**: Holds configuration settings like database credentials and security options.
- **Blueprints (**farmer_bp, customer_bp, payment_bp**):** These separate different parts of the app for better modularity.
- **Common functions (**home, register, login, profile, logout**)**: Handle authentication and home page functionality.
- **datetimeformat:** A custom Jinja filter for formatting dates in templates.

2. **Initializing the Flask Application**

```
app = Flask(__name__)
```

```
app.config.from_object(Config)
```

- Creates an instance of the Flask application.
- Loads configuration settings from the Config class.

3. **Registering Blueprints** (Modular Routing)

```
app.register_blueprint(farmer_bp)

app.register_blueprint(customer_bp, url_prefix="/customer")

app.register_blueprint(payment_bp, url_prefix="/payment")
```

- Blueprints allow different parts of the app (farmer, customer, payment) to be managed separately.
- url_prefix="/customer" means that all routes inside customer_bp will start with /customer (e.g., /customer/orders).
- Similarly, payment_bp routes will start with /payment.

4. **Adding a Jinja Filter**

```
app.jinja_env.filters['datetimeformat'] = datetimeformat
```

- Registers datetimeformat as a Jinja filter, allowing templates to format dates dynamically.

5. **Defining Main Routes**

```
@app.route("/")
def home_route():
    return home()
@app.route("/register", methods=["GET", "POST"])
def register_route():
    return register()
@app.route("/login", methods=["GET", "POST"])
def login_route():
    return login()
@app.route("/profile", methods=["GET", "POST"])
def profile_route():
    return profile()
```

```
@app.route("/logout")
def logout_route():
    return logout()
```

- These route functions define different URL endpoints.
- When users visit /, /register, /login, /profile, or /logout, the respective functions are called.
- methods=["GET", "POST"] allows handling both form submissions (POST) and page loads (GET).

6. **Running the Flask App**

```
if __name__ == "__main__":
    app.run(debug=True)
```

- Ensures the script runs only when executed directly.
- debug=True enables debug mode, which automatically restarts the server on code changes and provides detailed error logs.

Responsibilities and Functionalities of app.py

1. Application Initialization

- Creates a **Flask app instance**.
- Loads configuration settings from the Config class.

2. Route Management

- Defines core **user authentication** routes (/, /register, /login, /profile, /logout).
- Handles HTTP **GET and POST** requests for user interactions.

3. Blueprint Registration

- **Modular structure** with separate Blueprints for:
    - **Farmers (farmer_bp)** – Manages farmer-related operations.
    - **Customers (customer_bp)** – Handles customer actions like ordering.
    - **Payments (payment_bp)** – Integrates the **Cashfree Payment Gateway**.

4. Data Formatting for Frontend

- Registers datetimeformat as a **Jinja filter**, ensuring date formatting in templates.

5. Running the Application

- Ensures the Flask app starts when executed directly.
- Enables **debug mode** for easier development and debugging.

2. **Customer.py (Manages Customer Orders)**

Detailed Explanation of customer.py

This module **manages customer-related functionalities**, including **browsing vegetables, ordering, cart management, viewing order history, and checkout**. It uses **Flask Blueprint (customer_bp)** to keep the routes organized.

1. Blueprint Initialization

```
customer_bp = Blueprint("customer", __name__)
```

- A **Flask Blueprint** is created for customer-related routes (customer_bp).
- This allows the customer functionality to be modular and reusable.

```
from flask import Blueprint, request, render_template, redirect, url_for, session, flash

from bson.objectid import ObjectId

from datetime import datetime

from config import *

from db import  *
```

- **Flask modules**: Blueprint, request, render_template, redirect, url_for, session, and flash for handling web requests, authentication, and user notifications.
- **MongoDB (bson.objectid.ObjectId)**: Converts string-based object IDs into MongoDB's ObjectId format for querying documents.
- **datetime**: Used to timestamp orders.
- **Configuration files (config.py and db.py)**: Import database connections and settings.

2. Browse Vegetables (/browse_vegetables)

```
@customer_bp.route("/browse_vegetables", methods=["GET", "POST"])

def browse_vegetables():
```

- **Handles browsing vegetables and placing orders.**
- **Supports both GET and POST requests**:
    - GET: Displays available vegetables.
    - POST: Adds vegetables to the cart or places an order.

Step 1: Authentication Check

```
if 'username' not in session:

   flash("Please log in to continue.", "error")

   return redirect(url_for("login_route"))
```

- If the user is **not logged in**, they are redirected to the login page.

Step 2: Retrieve Customer Details

```
customer = customers_collection.find_one({"username": session['username']})
if not customer:
    flash("Customer not found.", "error")
    return redirect(url_for("login_route"))
```

- The system **fetches customer details** from customers_collection.
- If the customer does not exist, an error message is flashed.

Step 3: Fetch Available Vegetables

```
vegetables = list(vegetables_collection.find())
```

- Retrieves **all vegetables** from vegetables_collection.

Step 4: Handle User Actions (Add to Cart or Order)

```
if request.method == "POST":
    vegetable_id = request.form["vegetable_id"]
    action = request.form["action"]
    quantity = int(request.form.get("quantity", 1))
```

- Extracts vegetable_id, action, and quantity from the submitted form.

Action 1: Add to Cart

```
if action == "add_to_cart":
    cart = carts_collection.find_one({"customer_id": customer["_id"]})
```

- Checks if the customer already has a **cart**.
- If the cart **does not exist**, it creates one.

```
carts_collection.insert_one({
    "customer_id": customer["_id"],
    "customer_name": customer["username"],
    "vegetables": [{
        "vegetable_id": vegetable_id,
        "vegetable_name": vegetable_name,
        "price": vegetable_price,
        "photo_filename": vegetable_photo,
        "farmer_username": farmer_username,
        "quantity": quantity
```

```
    }]
})
```

- If the cart **exists**, it adds the selected vegetable:

```
carts_collection.update_one(
    {"customer_id": customer["_id"]},
    {"$addToSet": {"vegetables": {
        "vegetable_id": vegetable_id,
        "vegetable_name": vegetable_name,
        "price": vegetable_price,
        "photo_filename": vegetable_photo,
        "farmer_username": farmer_username,
        "quantity": quantity
    }}}
)
```

- A **success message** is flashed and the page is refreshed.

Action 2: Place Order

```
elif action == "order":
    history_collection.insert_one({
        "customer_id": customer["_id"],
        "customer_name": customer["username"],
        "customer_phone": customer.get("phone", "N/A"),
        "farmer_phone": farmer_phone,
        "vegetable_id": vegetable_id,
        "vegetable_name": vegetable_name,
        "cost": vegetable_price * quantity,
        "photo_filename": vegetable_photo,
        "farmer_username": farmer_username,
        "status": "Ordered",
        "quantity": quantity,
        "order_time": datetime.now().timestamp()
    })
```

- Stores the **order details** in history_collection.
- Redirects the user to the **payment gateway**.

3. View Order History (/order_history)

```
@customer_bp.route("/order_history")
```

- Fetches all past orders from history_collection for the logged-in customer.
- Orders are sorted in **descending order** (newest first).

4. View Cart (/view_cart)

```
@customer_bp.route("/view_cart", methods=["GET", "POST"])
```

Step 1: Retrieve Cart

```
cart = carts_collection.find_one({"customer_id": customer["_id"]}) or {"vegetables": []}

total_price = sum(item["price"] * item["quantity"] for item in cart["vegetables"]) if cart["vegetables"] else 0
```

- Retrieves the **customer's cart** and calculates the total price.

Step 2: Handle Cart Actions

- **Update Quantity**

```
carts_collection.update_one(
    {"customer_id": customer["_id"], "vegetables.vegetable_id": vegetable_id},
    {"$set": {"vegetables.$.quantity": new_quantity}}
)
```

- **Remove Item**

```
carts_collection.update_one(
    {"customer_id": customer["_id"]},
    {"$pull": {"vegetables": {"vegetable_id": vegetable_id}}}
)
```

- **Proceed to Checkout**

```
elif action == "checkout":
    return redirect(url_for("customer.checkout"))
```

5. Checkout (/checkout)

```
@customer_bp.route("/checkout", methods=["POST"])
```

Step 1: Validate Cart

```
if not cart["vegetables"]:
```

```
    flash("Your cart is empty.", "error")
    return redirect(url_for("customer.view_cart"))
```

- If the cart is **empty**, the user is redirected.

Step 2: Store Order Details

```
history_collection.insert_one({
    "customer_id": customer["_id"],
    "customer_name": customer["username"],
    "customer_phone": customer.get("phone", "N/A"),
    "farmer_phones": farmer_phone_map,
    "order_items": vegetables_list,
    "cost": total_cost,
    "status": "Ordered",
    "order_time": datetime.now().timestamp()
})
```

- The **order is saved**, and the cart is **cleared**.

6. Clear Orders

Clear Selected Orders (/clear_selected_orders)

```
@customer_bp.route("/clear_selected_orders", methods=["POST"])
```

- Deletes **only selected orders** from history_collection.

Clear All Orders (/clear_all_orders)

```
@customer_bp.route("/clear_all_orders", methods=["POST"])
```

- Deletes **all order history** for the logged-in customer.

Responsibilities and Functionalities of customer.py

The customer.py module handles all customer-related operations in the system. It is responsible for managing vegetable browsing, shopping cart, orders, order history, and checkout process. It also ensures session management, order tracking, and communication with the database.

Responsibilities

1. **User Authentication & Session Management**
    - Ensures that only logged-in customers can browse, add to cart, and place orders.

- Redirects unauthorized users to the login page.

2. **Vegetable Browsing & Selection**
   - Retrieves the list of available vegetables from the database.
   - Allows customers to view vegetable details before purchasing.
3. **Cart Management**
   - Adds selected vegetables to the cart.
   - Updates the quantity of items in the cart.
   - Allows customers to remove items from the cart.
4. **Order Placement & Tracking**
   - Enables customers to place orders directly from the browse vegetables page.
   - Stores order details in the order history.
   - Assigns the order to the respective farmer.
5. **Order History & Management**
   - Displays past orders for a logged-in customer.
   - Allows customers to delete selected orders or clear all order history.
6. **Checkout & Payment Integration**
   - Calculates the total price of items in the cart.
   - Redirects the customer to the payment gateway for payment processing.
7. **Database Operations**
   - Interacts with MongoDB collections:
     - customers_collection → Retrieves customer data.
     - vegetables_collection → Fetches vegetable details.
     - carts_collection → Manages cart items.
     - history_collection → Stores order details.

Functionalities

Feature	Functionality
Browse Vegetables	Displays available vegetables and allows customers to select items.
Add to Cart	Customers can add vegetables to their cart with specified quantity.
View Cart	Displays items added to the cart with total price calculation.
Update Cart	Customers can modify the quantity of items or remove them.
Place Order	Customers can order vegetables directly, storing order details.
Order History	Displays past orders sorted by order date.

Clear Orders	Customers can delete selected or all orders from history.
Checkout	Calculates total cost, stores order details, and redirects to payment.

This module ensures smooth order processing, secure transactions, and user-friendly shopping experience.

3. Farmer.py (Manages Farmer Dashboard)

The farmer.py module handles **all functionalities related to farmers**, such as **managing vegetables, adjusting prices, removing products, viewing orders, and updating order statuses**. It interacts with MongoDB to store and retrieve data.

1. Importing Required Modules

```
from flask import Blueprint, render_template, request, redirect, url_for, flash, session

from werkzeug.utils import secure_filename

from bson import ObjectId

import os

from db import farmers_collection, vegetables_collection, history_collection

from config import Config

from common import datetimeformat
```

- **Flask Modules**: Used for creating routes, handling requests, and managing user sessions.
- **MongoDB (bson.ObjectId)**: Used for querying and updating MongoDB documents.
- **secure_filename**: Ensures safe file uploads for vegetable images.
- **Config & common**: Configuration settings and a date formatting helper function.

2. Blueprint Setup

```
farmer_bp = Blueprint('farmer', __name__)
```

- Registers this module as a **Blueprint** to integrate it into the main Flask application.

3. Helper Function: Allowed File Extensions

```
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in Config.ALLOWED_EXTENSIONS
```

- Ensures only **allowed image formats** (e.g., JPG, PNG) are uploaded.

4. Farmer Dashboard

```
@farmer_bp.route("/farmer_dashboard/<username>", methods=["GET", "POST"])
def farmer_dashboard(username):
    farmer = farmers_collection.find_one({"username": username})
    vegetables = vegetables_collection.find({"farmer_username": username})
```

- **GET Request**: Fetches farmer details and their listed vegetables.
- **POST Request**: Handles either:
  - **Updating vegetable prices**
  - **Adding a new vegetable (with optional image upload)**

5. Adjust Vegetable Price

```
@farmer_bp.route("/adjust_price/<username>/<vegetable_id>", methods=["GET", "POST"])
def adjust_price(username, vegetable_id):
    vegetable = vegetables_collection.find_one({"_id": ObjectId(vegetable_id)})
```

- Fetches **specific vegetable details**.
- Updates **name, price, and image** if submitted.

6. Remove Vegetable

```
@farmer_bp.route("/remove_vegetable/<username>/<vegetable_id>", methods=["POST"])
def remove_vegetable(username, vegetable_id):
    vegetable = vegetables_collection.find_one({"_id": ObjectId(vegetable_id)})

    if vegetable.get("farmer_username") != username:
        flash("You cannot delete this vegetable.", "error")
        return redirect(url_for("farmer.farmer_dashboard", username=username))
```

- Ensures **only the farmer who owns the vegetable** can delete it.
- **Deletes the vegetable and its associated image** from storage.

7. Update Order Status

```
@farmer_bp.route('/update_status/<order_id>', methods=['POST'])
def update_status(order_id):
    new_status = request.form.get("status")
```

```
    if not new_status:
        flash("No status selected. Please try again.", "danger")
        return redirect(url_for('farmer.view_orders'))
```

- **Updates the status** of a specific order (e.g., **"In Progress"**, **"Shipped"**).
- **Flashes a success/error message** based on whether the update was successful.

8. View Orders

```
@farmer_bp.route("/view_orders")
def view_orders():
    if 'username' not in session:
        flash("Please log in to view orders.", "error")
        return redirect(url_for("login_route"))


    farmer_username = session['username']
    orders = list(history_collection.find({"farmer_username": farmer_username}).sort("order_time", -1))
```

- **Requires authentication** (redirects if not logged in).
- Fetches **all orders related to the logged-in farmer** and sorts them by order time.

Summary of Functionalities

Feature	Functionality
Dashboard	Shows farmer details and listed vegetables.
Add Vegetable	Allows farmers to upload new vegetables with price and image.
Update Price	Farmers can modify vegetable prices and images.
Remove Vegetable	Deletes a vegetable and removes its image from storage.
View Orders	Lists all orders associated with the farmer.
Update Order Status	Changes the status of an order (e.g., "Shipped").

This module ensures **smooth management of vegetable listings and order processing for farmers.**

Responsibilities and Functionalities of farmer.py

The farmer.py module is responsible for managing the **farmer-related operations** in the system. It provides routes for **managing vegetables, updating prices, handling orders, and removing listings.**

◈ Responsibilities

The farmer.py module has the following key responsibilities:

1. **Farmer Authentication & Session Management**
   - Ensures that only authenticated farmers can access their dashboard and order history.
2. **Vegetable Management**
   - Allows farmers to **add, update, and remove** vegetables they sell.
   - Supports **image uploads** for vegetables with format validation.
3. **Price & Stock Control**
   - Enables farmers to **adjust prices** of their listed vegetables.
4. **Order Management**
   - Displays **all orders placed** for the farmer's vegetables.
   - Allows farmers to **update the status** of their orders (e.g., "Pending", "Shipped").
5. **Security & Access Control**
   - Ensures **only the rightful farmer** can modify or remove their products.
   - Prevents unauthorized actions through **session checks and ownership verification**.

Functionalities

1. Farmer Dashboard (/farmer_dashboard/<username>)

- Displays the farmer's information and vegetable listings.
- Allows the farmer to add a new vegetable with a price and image.
- Enables updating vegetable prices directly from the dashboard.

2. Adjust Vegetable Price (/adjust_price/<username>/<vegetable_id>)

- Fetches a specific vegetable's details for editing.
- Allows the farmer to update the name, price, and image of a vegetable.

3. Remove Vegetable (/remove_vegetable/<username>/<vegetable_id>)

- Ensures only the farmer who owns the vegetable can delete it.
- Deletes the vegetable record from the database.
- Removes associated image file from storage.

4. View Orders (/view_orders)

- Ensures the farmer is logged in before accessing orders.
- Retrieves all orders placed for the farmer's vegetables.
- Displays orders sorted by latest first for better visibility.

5.Update Order Status (/update_status/<order_id>)

- Allows the farmer to change the order status (e.g., "Shipped").
- Ensures the order exists before updating.
- Provides error handling in case of missing or invalid status updates.

Summary Table

Feature	Functionality
Dashboard	Displays farmer info and vegetable listings.
Add Vegetable	Allows farmers to add new vegetables with images.
Update Price	Enables price adjustments for existing vegetables.
Remove Vegetable	Farmers can delete vegetables they no longer sell.
View Orders	Shows all orders placed for the farmer's products.
Update Order Status	Farmers can mark orders as "Shipped", "Delivered", etc.

This module ensures **smooth and secure management of vegetable listings and orders for farmers.**

4. Payment_gateway.py (Handles Payments via Cashfree API)

Here's a detailed explanation of the payment.py file:

1. Importing Required Modules

```
from flask import Blueprint, request, redirect, url_for, session, flash, render_template

import uuid

import logging

from datetime import datetime

import requests

from config import Config

from db import customers_collection, history_collection, records_collection
```

- Flask Imports:
    - Blueprint: Used to create a modular component (payment_bp) for handling payment-related routes.
    - request: To handle incoming HTTP requests.
    - redirect, url_for: To redirect users to different routes.
    - session: To store user session data.
    - flash: To show temporary messages (e.g., errors, success notifications).

    - render_template: To render HTML templates.
- Other Imports:
    - uuid: To generate unique order IDs.
    - logging: To log errors for debugging.
    - datetime: To store timestamps in MongoDB.
    - requests: To make HTTP requests to the Cashfree payment gateway.
    - Config: Imports app configurations, including API credentials.
    - db: Imports MongoDB collections for customers, order history, and payment records.

2. Setting Up Payment Blueprint

payment_bp = Blueprint("payment", __name__)

- Creates a Flask Blueprint named payment_bp to group all payment-related routes.

3. Configuring Cashfree Payment Gateway

CASHFREE_BASE_URL = "https://sandbox.cashfree.com/pg/orders" if Config.CASHFREE_ENVIRONMENT == "sandbox" else "https://api.cashfree.com/pg/orders"

```
HEADERS = {
    "Content-Type": "application/json",
    "x-client-id": Config.CASHFREE_CLIENT_ID,
    "x-client-secret": Config.CASHFREE_CLIENT_SECRET,
    "x-api-version": "2023-08-01"
}
```

- Determines whether the app is in sandbox mode (test mode) or live mode based on Config.CASHFREE_ENVIRONMENT.
- Headers are set for Cashfree API requests using API keys from Config.

4. Payment Gateway Page

@payment_bp.route("/gateway")

def gateway():

- Purpose: Displays the payment gateway page where the customer sees their latest order details before proceeding with the payment.

```
    if 'username' not in session:
        flash("Please log in to continue.", "error")
        return redirect(url_for("login_route"))
```

- Checks if the user is logged in; if not, redirects to the login page.

```
    customer = customers_collection.find_one({"username": session['username']})
    if not customer:
        flash("Customer not found.", "error")
        return redirect(url_for("customer.browse_vegetables"))
```

- Retrieves customer details from MongoDB. If the customer is not found, an error message is displayed.

```
    latest_order = history_collection.find_one({"customer_id": customer["_id"]}, sort=[("order_time", -1)])
    total_amount = latest_order["cost"] if latest_order else 1.00
```

- Fetches the latest order for the customer.
- If no order exists, a default amount of ₹1.00 is set.

```
    return render_template(
        "gateway.html",
        customer_name=customer.get("name", customer["username"]),
        customer_email=customer.get("email", ""),
        customer_phone=customer.get("phone", ""),
        address=customer.get("address", ""),
        city=customer.get("city", ""),
        state=customer.get("state", ""),
        pincode=customer.get("pincode", ""),
        total_amount=total_amount
    )
```

- Renders the payment page (gateway.html) with customer details and the total amount.

5. Create Order Route

```
@payment_bp.route('/create-order', methods=['POST'])
def create_order():
```

- Purpose: Creates a new order and interacts with the Cashfree API to generate a payment session.

```
if 'username' not in session:
    flash("Please log in to continue.", "error")
    return redirect(url_for("login_route"))
```

- Ensures that the user is logged in before proceeding.

```
customer = customers_collection.find_one({"username": session['username']})
if not customer:
    flash("Customer not found.", "error")
    return redirect(url_for("customer.browse_vegetables"))
```

- Retrieves customer details.

```
latest_order = history_collection.find_one({"customer_id": customer["_id"]}, sort=[("order_time", -1)])
order_amount = latest_order.get("cost", 1.00) if latest_order else 1.00
```

- Fetches the latest order cost.

```
order_id = str(uuid.uuid4())
```

- Generates a unique order ID.

```
payload = {
    "order_id": order_id,
    "order_amount": order_amount,
    "order_currency": "INR",
    "customer_details": {
        "customer_id": str(customer["_id"]),
        "customer_name": customer_name,
        "customer_email": customer_email,
        "customer_phone": customer_phone
    },
    "order_meta": {
        "return_url": url_for("payment.payment_status", order_id=order_id, _external=True)
    }
}
```

- Payload for creating an order in the Cashfree payment gateway.

```
response = requests.post(CASHFREE_BASE_URL, json=payload, headers=HEADERS)
```

- Sends request to the Cashfree API.

```
if response.status_code == 200:
    data = response.json()
    payment_session_id = data.get("payment_session_id")
```

- If the response is successful, retrieves the payment session ID.

```
    records_collection.insert_one({
        "order_id": order_id,
        "customer_id": str(customer["_id"]),
        "customer_name": customer_name,
        "customer_email": customer_email,
        "customer_phone": customer_phone,
        "order_amount": order_amount,
        "payment_status": "PENDING",
        "created_at": datetime.utcnow()
    })
```

- Stores order details in MongoDB with PENDING status.

```
    if payment_session_id:
        return render_template("checkout.html", payment_session_id=payment_session_id,
order_amount=order_amount)
```

- Redirects to the checkout page with the session ID.

6. Payment Status Route

```
@payment_bp.route('/payment-status')
def payment_status():
```

- Purpose: Checks the status of an order from the Cashfree API.

```
    order_id = request.args.get("order_id")
    if not order_id:
        flash("Order ID is missing.", "error")
        return redirect(url_for("browse_vegetables"))
```

- Retrieves order_id from the request. If missing, an error is shown.

```
order = records_collection.find_one({"order_id": order_id})
if not order:
    flash("Order not found.", "error")
    return redirect(url_for("browse_vegetables"))
```

- Fetches the order details from MongoDB.

```
response = requests.get(f"{CASHFREE_BASE_URL}/{order_id}", headers=HEADERS)
```

- Sends a GET request to Cashfree API to check the order status.

```
if response.status_code == 200:
    data = response.json()
    order_status = data.get("order_status", "").upper()
```

- Parses the order status.

```
status_map = {
    "PAID": "Success",
    "ACTIVE": "Pending Payment",
    "FAILED": "Payment Failed",
    "EXPIRED": "Payment Expired",
    "CANCELLED": "Payment Cancelled"
}
user_friendly_status = status_map.get(order_status, "Unknown Status")
```

- Maps Cashfree statuses to user-friendly messages.

```
records_collection.update_one({"order_id": order_id}, {"$set": {"payment_status": user_friendly_status}})
```

- Updates the order status in MongoDB.

```
return render_template("payment_status.html", order=updated_order)
```

- Displays the payment status page.

This was the detailed explanation. Let me know when you want the responsibilities and functionalities.

Responsibilities and Functionalities of payment.py

Responsibilities:

1. **Handle Payment Flow**

- Manages the entire payment process, from order creation to payment confirmation.

2. **User Authentication & Session Management**
   - Ensures that only logged-in users can access payment-related routes.
3. **Order Creation & Storage**
   - Retrieves customer details and generates unique order IDs.
   - Stores order details in the records_collection database.
4. **Integration with Cashfree API**
   - Communicates with Cashfree to create orders and fetch payment statuses.
5. **Transaction Status Updates**
   - Updates the payment status in MongoDB based on API responses.
6. **User Feedback & Navigation**
   - Redirects users appropriately in case of missing data or errors.
   - Displays confirmation pages and error messages.

Functionalities:

1. **gateway()**
   - Displays the payment gateway page with customer details.
   - Fetches the latest order cost for the user.
2. **create_order()**
   - Generates a new order.
   - Sends an API request to Cashfree to initiate payment.
   - Stores order details in MongoDB.
   - Redirects to the checkout page with a payment session.
3. **payment_status()**
   - Retrieves the order status from the Cashfree API.
   - Updates the payment status in MongoDB.
   - Displays the payment status to the user.

5. **Db.py (Handles MongoDB Operations)**

Detailed Explanation of db.py

This file is responsible for setting up the MongoDB connection and defining the collections used in the application.

Code Breakdown

1. Importing Dependencies

```
from pymongo import MongoClient

from config import Config
```

- **pymongo.MongoClient** → Used to connect to the MongoDB database.
- **Config** → Imports database configuration settings (e.g., MongoDB connection URI) from config.py.

2. Establishing a Connection with MongoDB

```
client = MongoClient(Config.MONGO_URI)
```

- Creates a **MongoDB client** using the connection URI stored in Config.MONGO_URI.
- This allows the application to communicate with the MongoDB database.

3. Selecting the Database

```
db = client.get_database('farm_fresh')
```

- Connects to the **farm_fresh** database.
- If the database doesn't exist, MongoDB will create it automatically.

4. Defining Collections

```
farmers_collection = db.get_collection('farmers')

customers_collection = db.get_collection('customers')

vegetables_collection = db.get_collection('vegetables')

carts_collection = db.get_collection('carts')

history_collection = db.get_collection('history')

records_collection = db.get_collection('records')
```

Each collection represents a different **data entity** in the application:

- **farmers_collection** → Stores farmer details (e.g., username, contact info, etc.).

- **customers_collection** → Stores customer details (e.g., name, address, payment history, etc.).
- **vegetables_collection** → Stores information about vegetables (e.g., name, price, farmer details, etc.).
- **carts_collection** → Stores items added to a customer's cart before checkout.
- **history_collection** → Stores the order history of customers and farmers.
- **records_collection** → Stores payment records and transaction statuses.

Purpose of db.py

- Establishes a **single connection** to MongoDB that is used throughout the app.
- Defines **consistent access** to collections.
- Provides a **centralized database configuration**, making it easy to update the database connection.

This file is crucial as it ensures smooth communication between the application and the database. Let me know if you need any modifications!

Responsibilities and Functionalities of db.py

Responsibilities:

1. **Database Connection Management**
   - Establishes a connection to the MongoDB database using the URI from config.py.
   - Ensures that the connection remains available throughout the application's lifecycle.
2. **Collection Definition**
   - Defines and initializes collections for different entities (farmers, customers, vegetables, etc.).
   - Provides a structured way to access and manipulate data.
3. **Centralized Database Configuration**
   - Centralizes database connection settings, making it easier to modify or switch databases if needed.
   - Prevents redundant connection code across multiple files.

Functionalities:

1. **Creates a Connection to MongoDB**
    - Uses MongoClient to connect to MongoDB using the MONGO_URI from config.py.
2. **Accesses the farm_fresh Database**
    - Retrieves a reference to the **farm_fresh** database.
3. **Defines and Manages Collections**
    - Provides structured access to collections (farmers, customers, vegetables, etc.).
    - Allows other modules to perform CRUD operations on these collections.
4. **Supports Data Storage for Key Functional Areas**
    - **Farmers Collection** → Stores farmer details.
    - **Customers Collection** → Manages customer data.
    - **Vegetables Collection** → Maintains inventory of available vegetables.
    - **Carts Collection** → Handles temporary cart data before checkout.
    - **History Collection** → Records past orders.
    - **Records Collection** → Stores payment transactions.

This file acts as the **database backbone** of the application, ensuring smooth and structured interactions with MongoDB.

6. Config.py (Stores Configuration Settings)

Detailed Explanation of config.py

The config.py file is responsible for storing the application's configuration settings, including Flask-specific settings, MongoDB connection details, and Cashfree API credentials.

Code Breakdown:

1. Importing the os module

import os

- The os module is used to interact with the operating system.
- It helps retrieve environment variables securely to avoid hardcoding sensitive information.

2. Defining the Config Class

class Config:

- The Config class centralizes all configuration settings for the application.
- This class is commonly used in Flask apps for easy access to environment-specific configurations.

3. General Flask Configuration

SECRET_KEY = os.environ.get('SECRET_KEY', 'your_default_secret_key')

- The **SECRET_KEY** is essential for securing Flask sessions, cookies, and CSRF protection.
- It is retrieved from the environment variables, with a default fallback value ('your_default_secret_key').

SESSION_COOKIE_NAME = 'your_session_cookie_name'

- Specifies the name of the session cookie used for tracking user sessions.

UPLOAD_FOLDER = 'static/uploads'

- Defines the **directory** where uploaded files (such as images) will be stored.

ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

- Defines a **set of allowed image file extensions** to restrict the type of files that can be uploaded.

DEBUG = os.environ.get('DEBUG', True)

- Controls whether Flask runs in **debug mode** (True) or **production mode** (False).
- When True, errors are displayed in the browser for easier debugging.

4. MongoDB Configuration

MONGO_URI = os.environ.get('MONGO_URI',

'mongodb://zubair:1234@cluster0-shard-00-00.phsy7.mongodb.net:27017,'

'cluster0-shard-00-01.phsy7.mongodb.net:27017,'

'cluster0-shard-00-02.phsy7.mongodb.net:27017/farm_fresh?ssl=true&replicaSet=atlas-numoic-shard-0&authSource=admin&retryWrites=true&w=majority'

)

- **MONGO_URI** is the **MongoDB connection string**, which connects to a remote MongoDB database.
- It includes:
    - **Username & Password:** zubair:1234
    - **Cluster Details:** Multiple shard URLs (cluster0-shard-00-00, cluster0-shard-00-01, etc.)

- **Database Name:** farm_fresh
- **Additional Parameters:**
  - ssl=true: Enables SSL encryption for secure communication.
  - replicaSet=atlas-numoic-shard-0: Uses a replica set for high availability.
  - authSource=admin: Authenticates against the admin database.
  - retryWrites=true&w=majority: Enables automatic retry for failed writes.

5. Cashfree Payment Gateway Configuration

CASHFREE_CLIENT_ID = os.environ.get('CASHFREE_CLIENT_ID', 'TEST10441503142baf2bc588bc0aed4b30514401')

- **Client ID** for integrating with the **Cashfree payment gateway**.
- Retrieved from the environment variables, with a test value as a fallback.

CASHFREE_CLIENT_SECRET = os.environ.get('CASHFREE_CLIENT_SECRET', 'cfsk_ma_test_6c339c4956bed590cbe5bdccbf82127a_6c39c6a4')

- **Client Secret Key** for authentication with the Cashfree API.

CASHFREE_ENVIRONMENT = os.environ.get('CASHFREE_ENVIRONMENT', 'sandbox')

- Determines whether the app runs in **sandbox mode (test transactions)** or **production mode (real transactions)**.
- Defaults to 'sandbox' to prevent accidental real transactions during development.

Summary

- **Security:** Uses environment variables for sensitive credentials.
- **Flask Settings:** Manages session security, file uploads, and debugging mode.
- **MongoDB Configuration:** Provides a connection string for database access.
- **Cashfree Payment API Settings:** Manages payment gateway credentials.

This file serves as the **centralized configuration hub** for the entire application.

Responsibilities and Functionalities of config.py

The config.py file plays a crucial role in configuring the application. Below are its key responsibilities and functionalities:

Responsibilities:

1. **Centralized Configuration Management**

- Acts as a single source for application settings, reducing hardcoding and duplication.

2. **Security and Environment Management**
   - Fetches sensitive credentials from environment variables instead of hardcoding them.
   - Ensures secure handling of **secret keys**, **database URIs**, and **API credentials**.
3. **Flask App Configuration**
   - Manages session security (SECRET_KEY).
   - Defines upload folder paths and allowed file extensions.
   - Enables or disables debugging mode.
4. **Database Configuration**
   - Provides the **MongoDB URI** for connecting to the database.
   - Ensures the app interacts with the correct database (farm_fresh).
5. **Payment Gateway Integration**
   - Stores **Cashfree API credentials** (CLIENT_ID, CLIENT_SECRET).
   - Specifies whether the app runs in **sandbox mode** (testing) or **production mode** (real transactions).

Functionalities:

1. **Secure Storage of Configuration Variables**
   - Prevents exposing sensitive data in the codebase by using environment variables.
2. **Session and Cookie Management**
   - Defines SECRET_KEY and SESSION_COOKIE_NAME for user session handling.
3. **File Upload Handling**
   - Specifies UPLOAD_FOLDER and ALLOWED_EXTENSIONS to manage file uploads.
4. **Database Connection Handling**
   - Provides MONGO_URI to establish a connection to the MongoDB database.
5. **Dynamic Environment Selection for Payments**
   - Allows switching between **sandbox** and **production** modes for the **Cashfree payment gateway**.

Why This Matters?

- **Easier Deployment**: Configuration values can be changed without modifying the code.
- **Security Best Practices**: Keeps sensitive data out of source code.
- **Scalability**: Simplifies environment management for development, testing, and production.

This file is **essential** for the smooth operation of the Flask application, ensuring all configurations are properly managed!

6. common.py(the common routes like home,logoutand profile)

Detailed Explanation of common.py

This file handles **authentication (registration, login, logout), user profile management, and session handling** in the Flask application.

1. Importing Required Modules

```
from flask import render_template, request, redirect, url_for, flash, session

from werkzeug.security import generate_password_hash, check_password_hash

import re

from datetime import datetime

from db import farmers_collection, customers_collection  # Import collections from db.py
```

- **Flask Modules**:
  - render_template: Renders HTML templates.
  - request: Accesses form data sent by users.
  - redirect, url_for: Redirects users to different routes.
  - flash: Displays success/error messages.
  - session: Manages user authentication state.
- **Werkzeug Security**:
  - generate_password_hash: Hashes passwords for security.
  - check_password_hash: Verifies hashed passwords during login.
- **Regex (re)**: Used to validate phone numbers.
- **Datetime (datetime)**: Converts timestamps into human-readable formats.
- **MongoDB Collections**: Imports farmers_collection and customers_collection from db.py.

2.Home Route

```
def home():
    return render_template("home.html")
```

- Renders the home.html template, which is likely the **landing page** for the application.

3. User Registration

```
def register():
    if request.method == "POST":
        username = request.form.get("username")
        password = request.form.get("password")
        phone = request.form.get("phone")
        role = request.form.get("role")
```

- Accepts **POST requests** with form inputs: username, password, phone, and role (farmer or customer).

◈ Validation Checks

```
        if not username or not password or not phone or not role:
            flash("All fields are required!", "error")
            return redirect(url_for("home_route"))

        if role not in ["farmer", "customer"]:
            flash("Invalid role selected!", "error")
            return redirect(url_for("home_route"))

        if not re.match(r"^\d{10}$", phone):
            flash("Phone number must be exactly 10 digits!", "error")
            return redirect(url_for("home_route"))
```

- **Ensures all fields are provided**.
- **Verifies role is valid (farmer or customer)**.
- **Validates phone number format (10 digits using regex)**.

◈ Storing User Data

```
        hashed_password = generate_password_hash(password)
        collection = farmers_collection if role == "farmer" else customers_collection
```

```
    if collection.find_one({"$or": [{"username": username}, {"phone": phone}]}):
        flash("Username or phone number already exists!", "error")
        return redirect(url_for("home_route"))

    collection.insert_one({
        "username": username,
        "password": hashed_password,
        "phone": phone,
        "role": role
    })

    flash(f"{role.capitalize()} registered successfully!", "success")
    return redirect(url_for("login_route"))
```

- **Hashes the password** before storing it in the database.
- **Chooses the correct collection (farmers or customers)** based on the role.
- **Checks if the username or phone number already exists** (prevents duplicates).
- **Inserts new user data** into MongoDB.
- **Flashes success message and redirects** to the login page.

4. User Login

```
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        role = request.form["role"]
```

- **Extracts form data** from login form (username, password, role).

◈ Validation Checks

```
        if not username or not password or not role:
            flash("All fields are required!", "error")
            return redirect(url_for("home_route"))
```

- Ensures all fields are provided.

◈ Authenticate User

```
collection = farmers_collection if role == "farmer" else customers_collection
user = collection.find_one({"username": username})

if user and check_password_hash(user["password"], password):
    session["username"] = username
    session["role"] = role
```

- **Fetches user details** from MongoDB based on role.
- **Verifies password** using check_password_hash().
- **Stores username & role in the session** (maintains login state).

◈ Redirect Users Based on Role

```
    if role == "farmer":
        return redirect(url_for("farmer.farmer_dashboard", username=username))
    else:
        return redirect(url_for("customer.browse_vegetables"))
```

- **Farmers** → Redirect to **farmer dashboard**.
- **Customers** → Redirect to **browse vegetables page**.

◈ Handle Invalid Credentials

```
else:
    flash("Invalid credentials! Please check your username, password, or role.", "error")
    return redirect(url_for("home_route"))
```

- If credentials are incorrect, **flash an error message and redirect**.

5. Profile Management

```
def profile():
  if 'username' not in session:
    flash("Please log in to view your profile.", "error")
    return redirect(url_for('login_route'))
```

- **Ensures user is logged in** before accessing profile.

◈ Fetch User Data

```
user = farmers_collection.find_one({"username": session['username']}) or \
    customers_collection.find_one({"username": session['username']})

if not user:
    flash("User not found.", "error")
    return redirect(url_for('login_route'))
```

- **Checks both collections** (farmers/customers) to retrieve user data.
- **Redirects to login if user is not found**.

◈ Update User Information

```
if request.method == "POST":
    new_username = request.form.get("username")
    new_password = request.form.get("password")
    new_phone = request.form.get("phone")

    collection = farmers_collection if user.get('role') == 'farmer' else customers_collection
    if new_username:
        collection.update_one({"username": session['username']}, {"$set": {"username": new_username}})
        session['username'] = new_username
        flash("Username updated successfully.", "success")
    if new_password:
        hashed_password = generate_password_hash(new_password)
        collection.update_one({"username": session['username']}, {"$set": {"password": hashed_password}})
        flash("Password updated successfully.", "success")
    if new_phone and re.match(r"^\d{10}$", new_phone):
        collection.update_one({"username": session['username']}, {"$set": {"phone": new_phone}})
        flash("Phone number updated successfully.", "success")
    else:
        flash("Invalid phone number.", "error")
```

- **Allows users to update** username, password, and phone.
- **Hashes new passwords** before storing them.

- **Updates session values** when the username changes.

◈ Return Updated Profile

```
user = farmers_collection.find_one({"username": session['username']}) or \
        customers_collection.find_one({"username": session['username']})


return render_template("profile.html", user=user)
```

- **Renders profile.html with updated user data**.

6.Order Time Formatter for Jinja

```
def datetimeformat(value, format='%Y-%m-%d %H:%M:%S'):
    if isinstance(value, (int, float)):
        return datetime.fromtimestamp(value).strftime(format)
    return value
```

- **Converts timestamps into human-readable dates**.
- **Used in Jinja templates** to display formatted order times.

7.Logout Function

```
def logout():
    session.clear()
    flash("You have been logged out.", "info")
    return redirect(url_for("login_route"))
```

- **Clears session data** (logs the user out).
- **Redirects to login page**.

Summary

- **Handles authentication** (register, login, logout).
- **Manages user profiles** (updates username, password, phone).
- **Ensures security with password hashing**.
- **Uses Flask sessions for authentication state**.
- **Validates inputs and prevents duplicate users**.
- **Formats timestamps for better readability**.

This module ensures **secure and efficient user management** in the application.

Responsibilities and Functionalities of common.py

The common.py file handles **core authentication and profile management** for farmers and customers. Below are its key responsibilities and functionalities:

Responsibilities:

1. **User Authentication**
   - Handles **registration, login, and logout** for both farmers and customers.
   - Ensures user passwords are securely hashed using werkzeug.security.
2. **Session Management**
   - Uses Flask sessions to maintain user login state (session["username"] and session["role"]).
   - Clears session data upon logout.
3. **Profile Management**
   - Allows users to update their **username, password, and phone number**.
   - Ensures phone numbers follow a **10-digit format**.
   - Updates session values when the username is changed.
4. **User Data Validation**
   - Prevents duplicate usernames or phone numbers during registration.
   - Ensures all fields are filled before processing login or registration.
5. **Role-Based Access Control**
   - Differentiates between **farmers and customers** using the role field.
   - Redirects users to appropriate dashboards after login.
6. **Custom Jinja Filter for Formatting Dates**
   - Provides a function (datetimeformat) to convert Unix timestamps into readable dates.

Functionalities:

1. **Home Route (home())**
   - Renders the **home page** of the application.
2. **User Registration (register())**
   - Collects **username, password, phone, and role** from the form.
   - Validates inputs, hashes passwords, and stores user details in **MongoDB**.
   - Redirects to the login page after successful registration.

3. **User Login (login())**
    - Validates credentials against **hashed passwords** in the database.
    - Stores user info in **session** upon successful login.
    - Redirects **farmers** to their **dashboard** and **customers** to the **vegetable browsing page**.
4. **Profile Management (profile())**
    - Fetches user details based on session data.
    - Allows updating **username, password, and phone number**.
    - Ensures phone numbers are **exactly 10 digits**.
    - Updates MongoDB and refreshes session values accordingly.
5. **Order Time Formatter (datetimeformat())**
    - Converts **Unix timestamps** into readable date formats for templates.
6. **User Logout (logout())**
    - Clears session data and redirects to the login page.

2. Why This Matters?

- **Enhances security** by hashing passwords.
- **Prevents unauthorized access** with session-based authentication.
- **Ensures data integrity** by enforcing unique usernames and phone numbers.
- **Improves user experience** by allowing profile updates.

This file is a **crucial part of user management**, ensuring authentication and profile handling are seamless!

System Module Overview

Farm Fresh System - Module Overview with Respective Modules

1. **Authentication & User Management** (*common.py*)
    - Handles user registration, login, profile updates, and session management for both farmers and customers.
    - Uses password hashing for security and maintains user records in MongoDB.
2. **Product & Cart Management** (*farmer.py, customer.py, db.py*)
    - Farmers can add, update, and delete vegetable listings.
    - Customers can browse, search, and add items to their cart.
    - Cart and product data are stored in MongoDB.

3. **Order & History Management** (*customer.py, db.py*)
    - Stores order details and tracks past purchases.
    - Farmers can view customer orders, and customers can track their history.
    - Maintains order history in MongoDB.
4. **Payment Processing** (*payment.py, config.py, db.py*)
    - Integrates Cashfree API for secure online transactions.
    - Generates order IDs, processes payments, and updates payment statuses.
    - Stores payment records in MongoDB.
5. **Farmer Dashboard & Management** (*farmer.py, db.py*)
    - Farmers can manage inventory, set prices, and track sales.
    - Provides an interface to access orders and transaction history.
6. **Customer Browsing & Shopping** (*customer.py, db.py*)
    - Customers can explore available vegetables, filter products, and proceed to checkout.
    - Orders are placed and recorded in the system.
7. **Session & Security Management** (*common.py, config.py*)
    - Ensures authentication, session security, and restricted access to authorized users.
    - Uses Flask sessions and environment-based configurations.
8. **Database & Configuration Management** (*db.py, config.py*)
    - Manages MongoDB connections and application settings.
    - Stores essential credentials and configurations for system operations.

5. Database Design

Farm Fresh Database Design Explanation

The database design of the **Farm Fresh System** consists of multiple collections within a MongoDB database. These collections manage user authentication, product details, order processing, cart management, and transaction records. Below is a detailed breakdown of each collection and its role in the system.

1. Database Schema Overview

The **Farm Fresh** database contains the following collections:

- **farmers**
- **customers**
- **vegetables**
- **carts**
- **history**
- **records**

Each collection is designed to handle a specific aspect of the application. Below is a detailed breakdown of each collection:

2. Farmers Collection

Purpose:

- Stores information about farmers who sell vegetables on the platform.

Fields:

Field Name	Description
id	Unique identifier for the farmer.
username	Farmer's chosen username.
password	Hashed password for authentication.
role	Role assigned as "farmer".
phone	Farmer's registered phone number.

Functionality:

- Farmers can **register** and **log in** using their credentials.
- They can **add vegetables** to the system.
- They can **update** the quantity, price, and photo of their products.

3. Customers Collection

Purpose:

- Stores information about customers who purchase vegetables.

Fields:

Field Name	Description
id	Unique identifier for the customer.
username	Customer's chosen username.
password	Hashed password for authentication.
role	Role assigned as "customer".
phone	Customer's registered phone number.

Functionality:

- Customers can **register** and **log in**.
- Customers can **browse vegetables** and add them to their cart.
- Customers can **place orders** and manage their order history.

4. Vegetables Collection

Purpose:

- Stores details of vegetables listed for sale by farmers.

Fields:

Field Name	Description
_id	Unique identifier for the vegetable.
vegetable_name	Name of the vegetable.
price	Price per unit of the vegetable.
photo_filename	Image file of the vegetable.
farmer_username	Farmer who listed the vegetable.

Functionality:

- Farmers can **add vegetables** to the platform.
- Farmers can **update** vegetable details (name, quantity, price, image).
- Customers can **view available vegetables** while shopping.

5. Carts Collection

Purpose:

- Stores the cart details of customers before checkout.

Fields:

Field Name	Description
_id	Unique identifier for the cart entry.
customer_id	Customer who added items to the cart.
customer_name	Customer's name.
vegetables	List of vegetables added to the cart.

Functionality:

- Customers can **add items** to their cart.
- Customers can **update item quantities** in the cart.
- The system allows **proceeding to checkout** and payment processing.

6. History Collection

Purpose:

- Stores records of past orders placed by customers.

Fields:

Field Name	Description
id	Unique identifier for the order.
customer_id	ID of the customer who placed the order.
customer_name	Name of the customer.
customer_phone	Phone number of the customer.
farmer_phone	Phone number of the farmer fulfilling the order.
vegetable_id	ID of the vegetable being purchased.
vegetable_name	Name of the vegetable.
cost	Cost of the vegetable.
photo_filename	Image of the vegetable.
farmer_username	Name of the farmer selling the vegetable.
status	Current status of the order (e.g., Pending, Shipped, Delivered).
quantity	Quantity of the vegetable purchased.

order_time	Timestamp of when the order was placed.

Functionality:

- When an order is placed, its details are stored in this collection.
- Farmers can **view orders** and update order statuses.
- Customers can **track order history** (items purchased, price, quantity, seller details).

7. Records Collection

Purpose:

- Stores details of orders related to payment processing.

Fields:

Field Name	Description
_id	Unique identifier for the transaction.
order_id	Associated order ID.
customer_name	Name of the customer.
customer_email	Email address of the customer.
customer_phone	Phone number of the customer.
order_amount	Total amount paid for the order.
payment_status	Status of the payment (e.g., Pending, Completed).
address	Shipping address.
city	City of the customer.
state	State of the customer.
pincode	Pincode of the customer.
created_at	Timestamp of when the order was created.

Functionality:

- Stores customer **delivery details**.
- Stores **payment status** (whether the payment was successful or pending).
- Helps in managing **order deliveries**

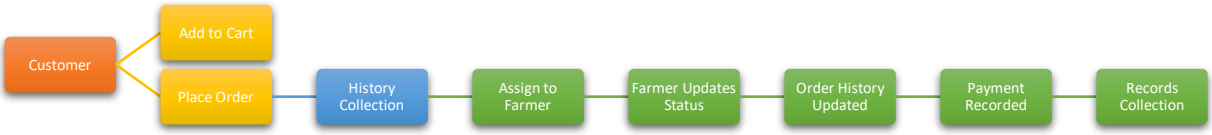
8. Data Flow Explanation

Data Flow Between Customers and Farmers:

1. **Customer Actions**:

- **Add Items to Cart**: The customer adds items to their cart. The system records this as part of the customer's session or temporary cart.
- **Place Orders**: When the customer is ready, they place an order. This triggers the creation of an order entry in the database.

2. **Order Storage**:
   - The order data (which includes customer details, cart items, etc.) is stored in the history_collection of the database. Each order is assigned to a specific farmer for fulfillment. The order status is initially set to something like "Pending" or "New."
3. **Farmer Actions**:
   - **Update Order Status**: The farmer can update the status of the order (e.g., "Processing," "Shipped," "Delivered"). These updates are saved back into the history_collection.
   - **Order History Reflection**: Whenever the status is updated, it will be reflected in the customer's order history, making it available for the customer to track.
4. **Payment Details**:
   - Once an order is confirmed, payment details are captured and stored in the records_collection to ensure that financial transactions are documented.



9. Summary

- **User Authentication:** Farmers and customers are stored in separate collections.
- **Product Management:** Farmers manage their inventory in the **vegetables collection**.
- **Order Processing:** Orders are stored in **history**, while carts track items before checkout.
- **Payment & Delivery:** Payment details are stored in **records**, ensuring smooth order tracking.
- **Scalability:** The schema is designed to handle multiple farmers, customers, and transactions efficiently.

This database design ensures efficient management of orders, inventory, payments, and user authentication while maintaining **data integrity and security**.

6. API Documentation

1. Overview

This API is designed for an online vegetable marketplace where farmers list vegetables, customers place orders, and payments are processed via **Cashfree Payment Gateway**. The API handles user authentication, vegetable browsing, cart management, payment processing, and order tracking.

2. Authentication & User Management

User Registration

- **Endpoint:** /register
- **Method:** POST
- **Description:** Registers a new user (farmer or customer).
- **Request Parameters:**

{ "username": "john_doe", "password": "securepassword", "role": "customer" }

- **Response:**

{ "message": "User registered successfully." }

User Login

- **Endpoint:** /login
- **Method:** POST
- **Description:** Authenticates a user and starts a session.
- **Request Parameters:**

{ "username": "john_doe", "password": "securepassword" }

- **Response:**

{ "message": "Login successful." }

User Logout

- **Endpoint:** /logout
- **Method:** GET
- **Description:** Ends the user session.

3. Vegetable & Order Management

Browse Vegetables

- **Endpoint:** /browse_vegetables
- **Method:** GET
- **Description:** Retrieves the list of available vegetables.

- **Response:**

[{ "name": "Tomato", "price": 50, "quantity": 10 }, { "name": "Potato", "price": 30, "quantity": 20 }]

Add to Cart

- **Endpoint:** /add_to_cart
- **Method:** POST
- **Description:** Adds a vegetable to the user's cart.
- **Request Parameters:**

{ "user_id": "12345", "vegetable_id": "67890", "quantity": 2 }

- **Response:**

{ "message": "Item added to cart." }

4. Payment Gateway (Cashfree)

Create Order

- **Endpoint:** /create_order
- **Method:** POST
- **Description:** Generates a payment order and provides a payment link.
- **Request Parameters:**

{ "user_id": "12345", "order_id": "ORDER001", "amount": 100.50 }

- **Response:**

{ "status": "success", "payment_link": "https://sandbox.cashfree.com/...ORD001" }

Fetch Payment Status

- **Endpoint:** /payment_status/<order_id>
- **Method:** GET
- **Description:** Retrieves the payment status from Cashfree.
- **Response:**

{ "order_id": "ORD001", "status": "PAID"}

Store Payment Details

- **Endpoint:** /store_payment
- **Method:** POST
- **Description:** Saves payment transaction details to MongoDB.
- **Request Parameters:**

{ "order_id": "ORD001", status": "PAID", "amount": 100.50 }

- **Response:**

{ "message": "Payment details stored successfully." }

5. Order & Delivery Management

Order Confirmation

- **Endpoint:** /order_details/<order_id>
- **Method:** GET
- **Description:** Fetches order details including payment status.
- **Response:**

{ "order_id": "ORD001", "customer_name": "John Doe", "amount": 100.50, "status": "PAID" }

Update Order Status

- **Endpoint:** /update_order_status
- **Method:** POST
- **Description:** Updates order status after payment confirmation.
- **Request Parameters:**

{ "order_id": "ORD001", "status": "Delivered" }

- **Response:**

{ "message": "Order status updated successfully." }

6. Error Handling

Common Errors:

Status Code	Description
400	Missing or invalid request parameters.
401	Unauthorized access.
404	Order or resource not found.
500	Internal server error.

Example Error Response:

{ "error": "Invalid order ID" }

7. Security Measures

- **Session-based authentication** for login/logout.
- **Validation checks** for API inputs.
- **Secure API keys** (stored in config.py).
- **Data encryption** for passwords using generate_password_hash().

7. Testing and Results

The system was tested using **dummy orders and payments** in a controlled sandbox environment to ensure that all API endpoints function correctly. The testing process covered various aspects of order creation, payment processing, database updates, and status retrieval. Below is a detailed breakdown of the testing process and results.

7.1 Testing Setup

1. **Environment:**
    - The system was deployed in a local Flask environment using **Termux**.
    - The **MongoDB Atlas cloud database** was used to store transaction details.
    - The **Cashfree sandbox API** was used to simulate real-world payment transactions.
2. **Test Data:**
    - **Dummy users** (both customers and farmers) were created.
    - **Vegetables** were listed in the database with mock pricing.
    - **Orders** were generated with unique order IDs and amounts.
    - **Simulated payments** were made using test card details provided by Cashfree.
    -

7.2 Test Cases and Observations

Test Case	Input	Expected Output	Actual Output	Result
1. User Registration	username: "test_user", password: "123456"	User is successfully registered	User was stored in MongoDB and redirected to login	✅ Passed
2. User Login	username: "test_user", password: "123456"	User logs in and session starts	Session stored and user redirected to dashboard	✅ Passed
3. Browse Vegetables	GET /browse_vegetables	List of vegetables retrieved	Successfully fetched from MongoDB	✅ Passed
4. Add to Cart	vegetable_id: "veg123", quantity: 2	Item added to cart	Entry created in the carts collection	✅ Passed
5. Create Order	user_id: "user123", amount: 200	Payment link generated	Order created, Cashfree returned payment URL	✅ Passed

6. Complete Payment	Clicking Cashfree payment link	Transaction successful	Status updated in MongoDB	✅ Passed
7. Fetch Payment Status	GET /payment_status/ORD001	Status = "PAID"	Correct transaction details retrieved	✅ Passed
8. Store Payment in Database	order_id: "ORD001", status: "PAID"	Payment recorded in MongoDB	Entry created in records_collection	✅ Passed
9. Order Confirmation	GET /order_details/ORD001	Order details retrieved	Correct details displayed on the confirmation page	✅ Passed
10. Update Order Status	POST /update_order_status	Status updated to "Delivered"	Order status changed in MongoDB	✅ Passed

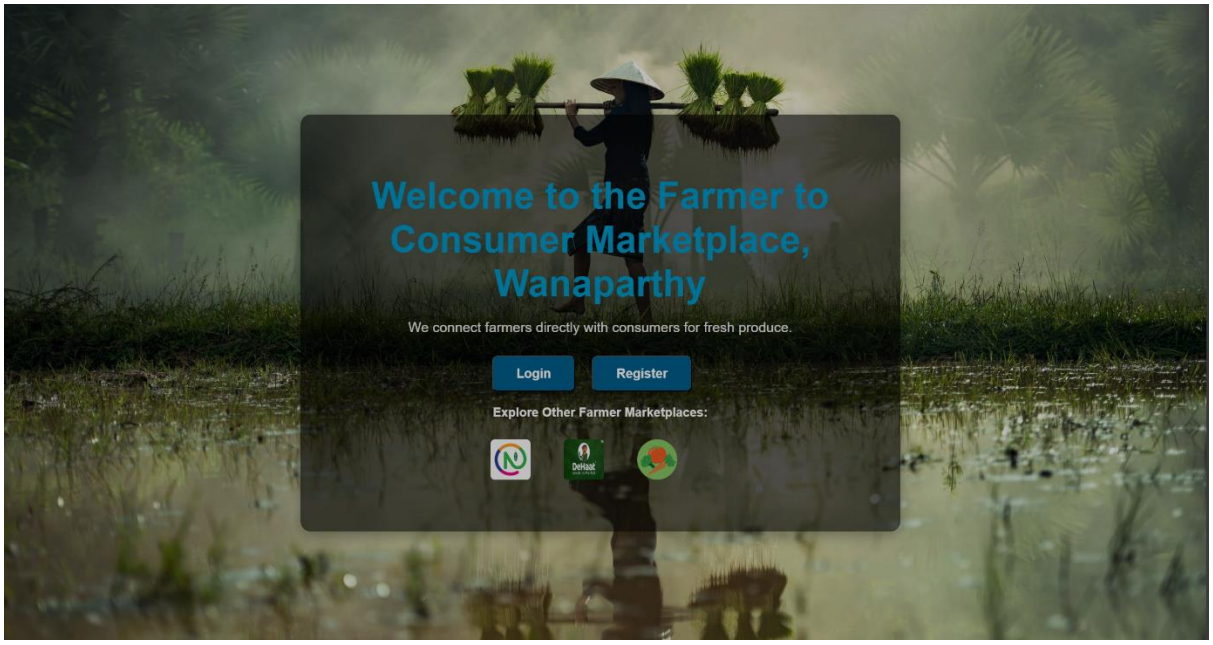
---

7.3 Observations

- **API Functionality:**
  - All API endpoints worked as expected in the sandbox mode.
  - Responses were consistent and formatted correctly in JSON.
- **Database Integrity:**
  - Data was correctly stored in **MongoDB Atlas**.
  - Transactions were accurately recorded in the history_collection and records_collection.
- **Payment Processing:**
  - Payment initiation via **Cashfree** was seamless.
  - Payment status retrieval correctly updated order records.
- **User Experience:**
  - Navigation between pages worked correctly.
  - Users could register, log in, browse vegetables, and place orders without issues.

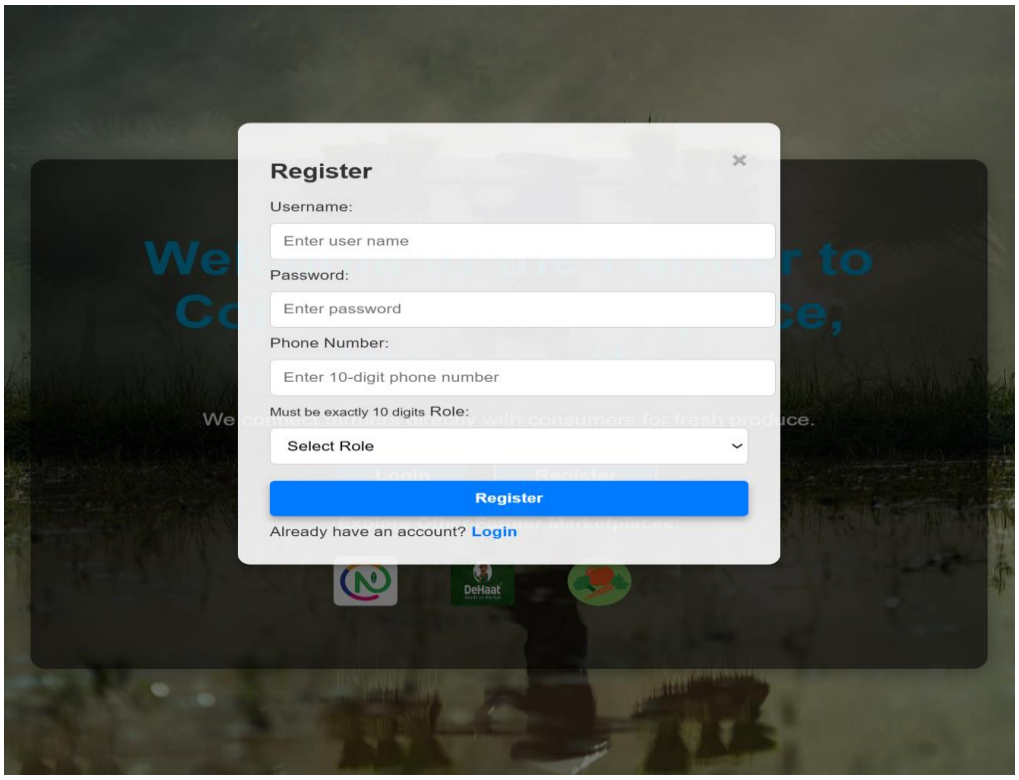
8.Application Demonstration

HOME PAGE:



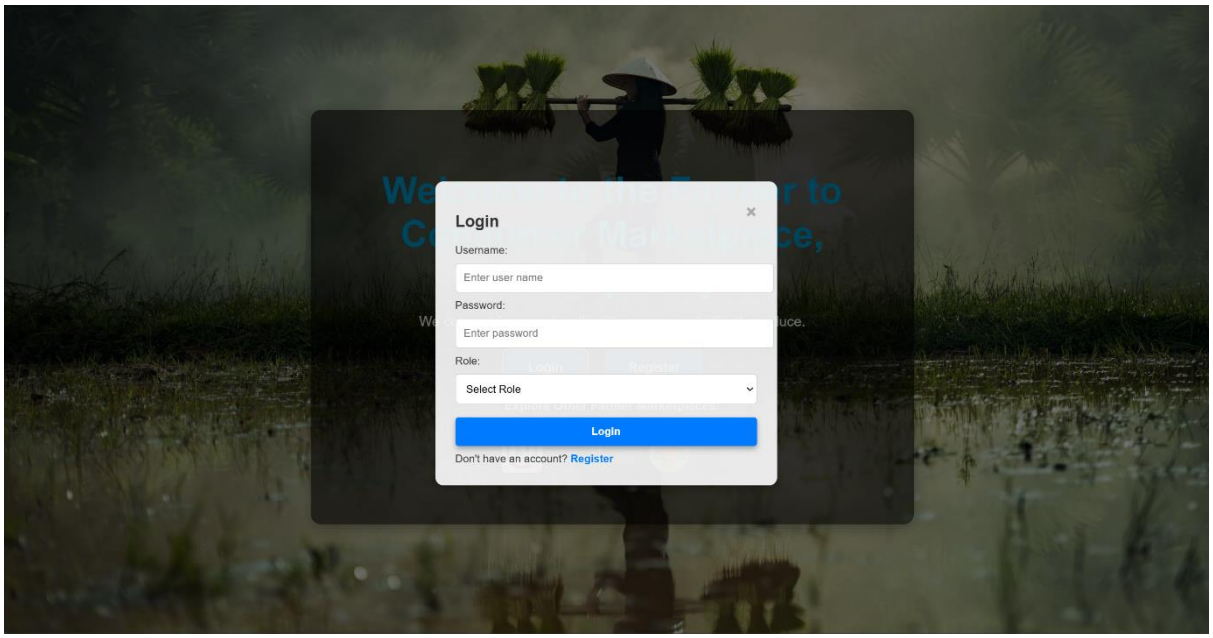
The above image is about the website **Farmer to Consumer Marketplace. In** this image we can see the home page of this website concluded with a **LOGIN** & **REGISTER** navigation button. we can see other websites which are similar to this website below the navigation buttons.

REGISTER PAGE:



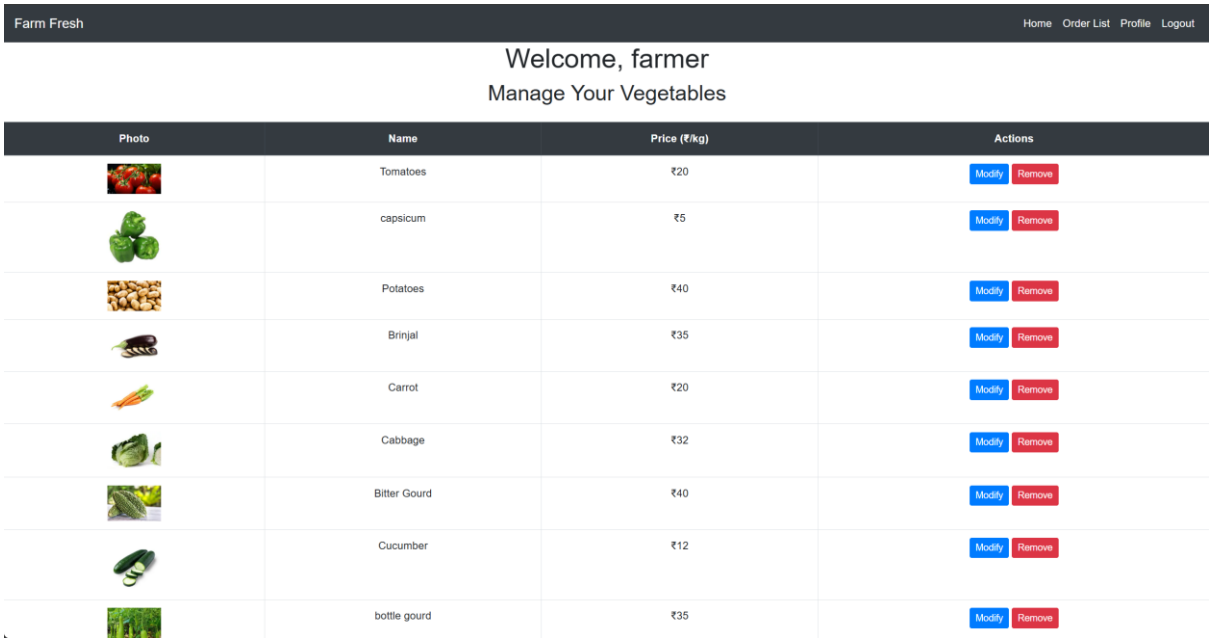
By clicking on register button, we can register an account of ours by selecting of our role as a **Farmer or Customer. As** we can see, enter the username ,password ,mobile number ,select role and register your account.

LOGIN PAGE:

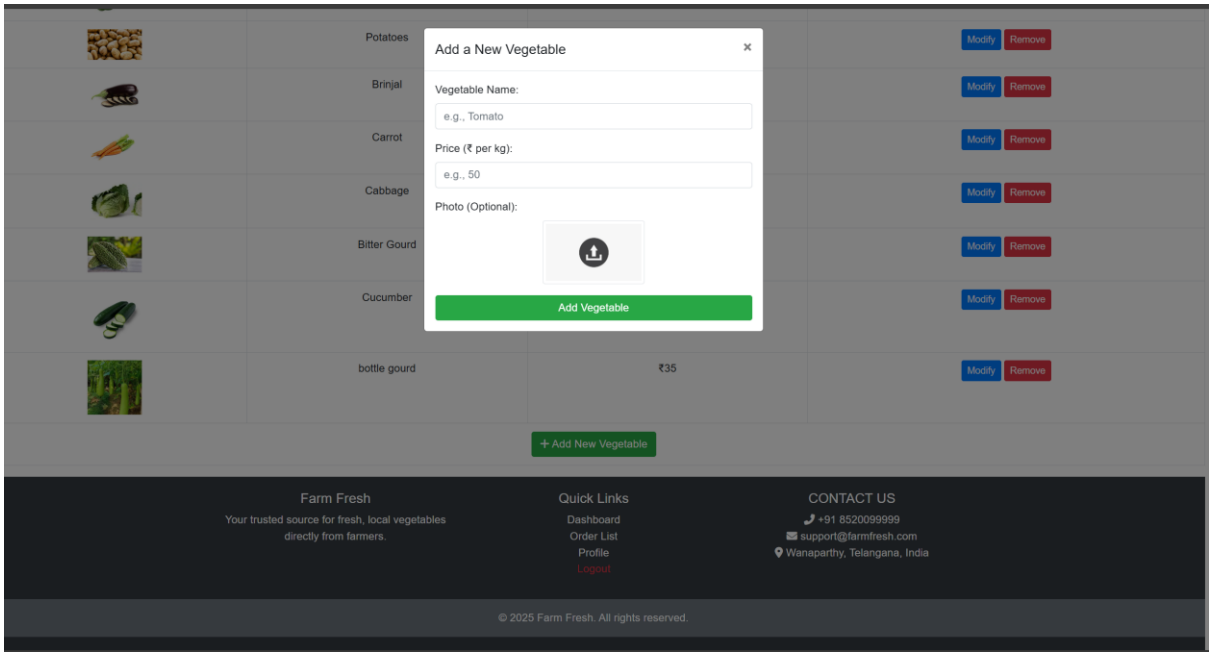


In this login page enter your details and select the role and click on login button .If you a don't have a account register by clicking on register.

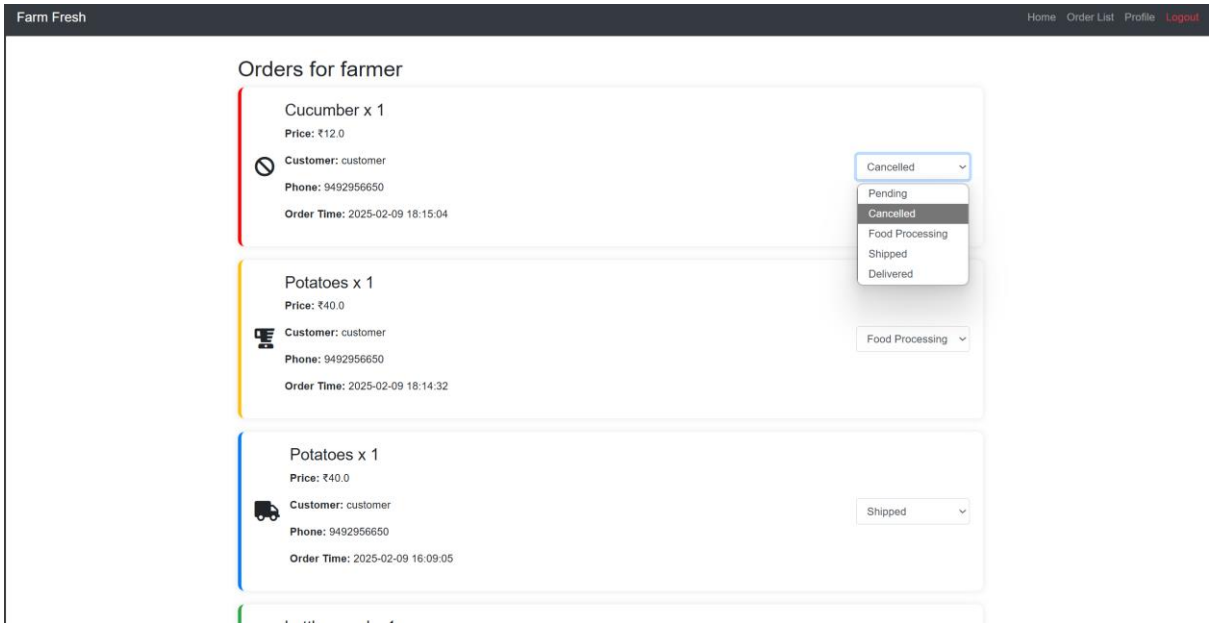
Home Page of Farmer:



In home page of farmer ,farmer can enter the details of available vegetables they have and can update their details of the vegetables .They can modify the prices according to the market price , they can remove the vegetables which they don't have the stock & can add new vegetables if they want.Image shown below

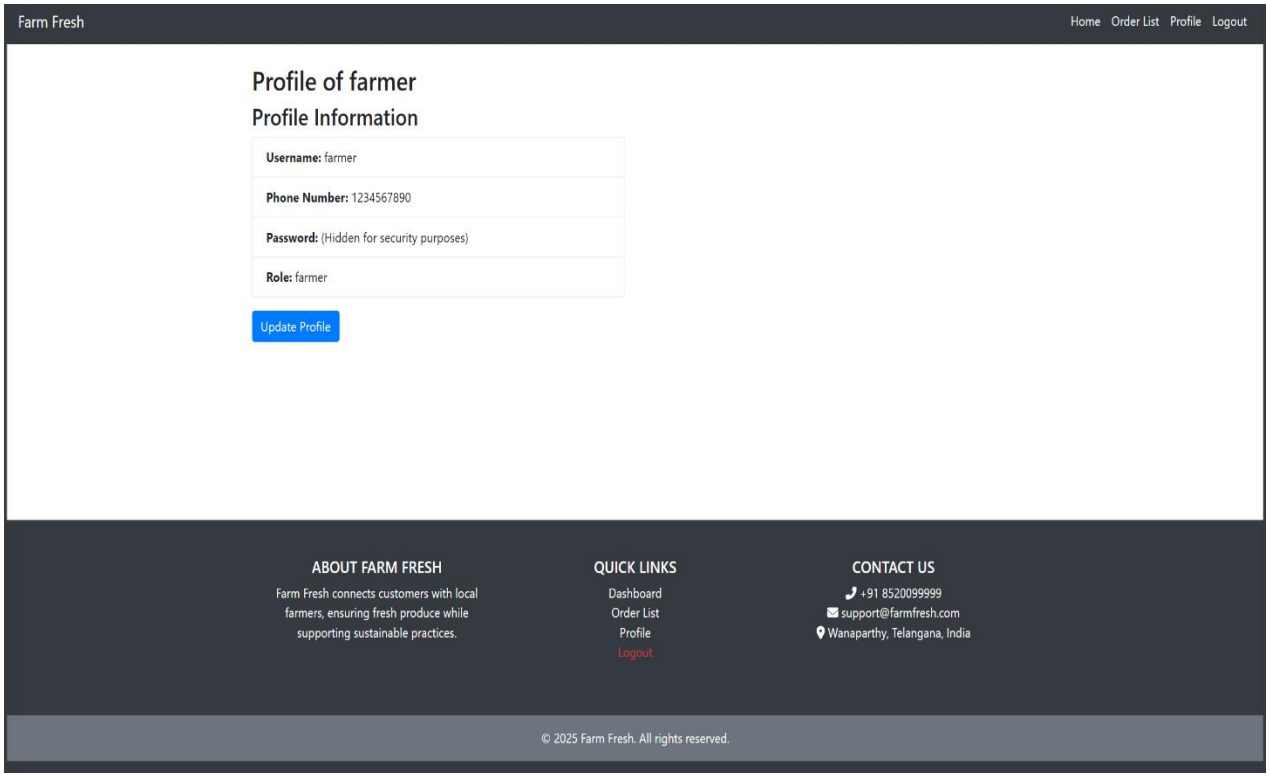


Order list of farmer:

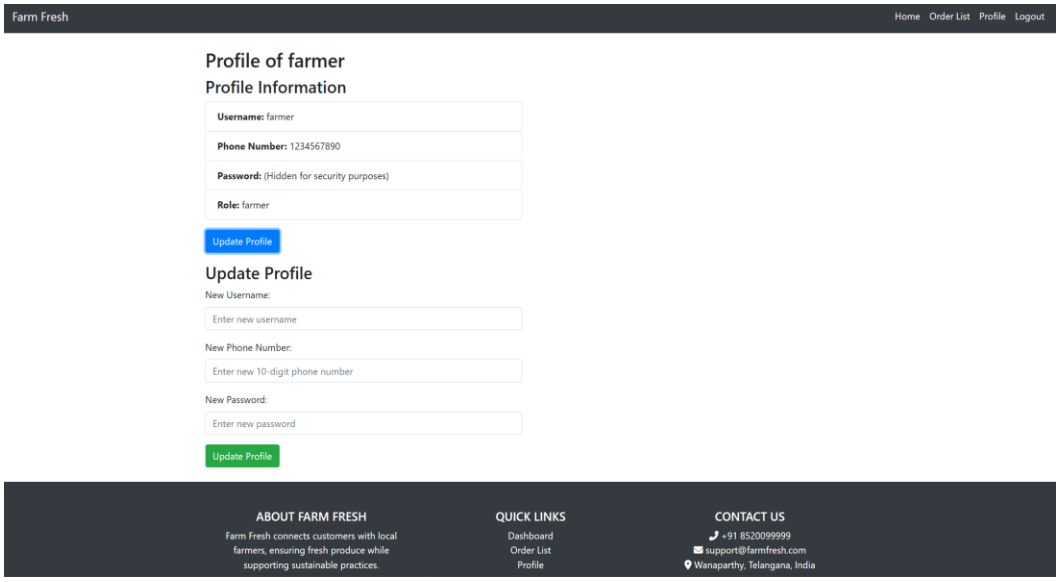


In this order list farmer can see the orders they got and contact the customers through mobile number and update the order status.

Profile of farmer:

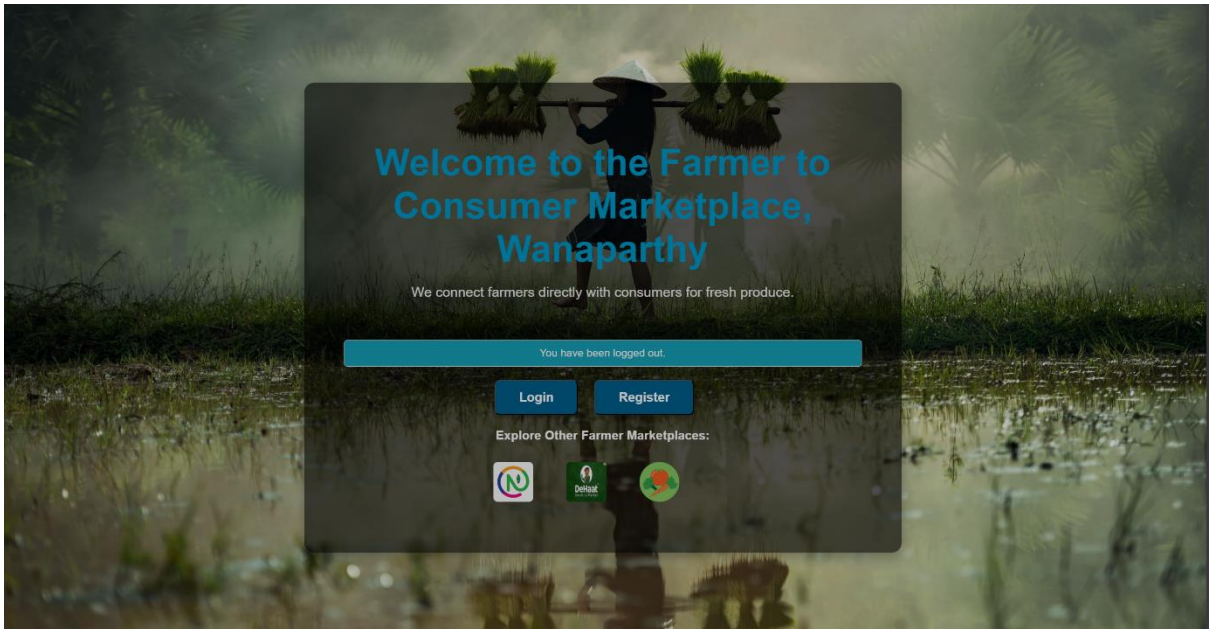


In this page we can update our profile by clicking on update profile, the image shown below.



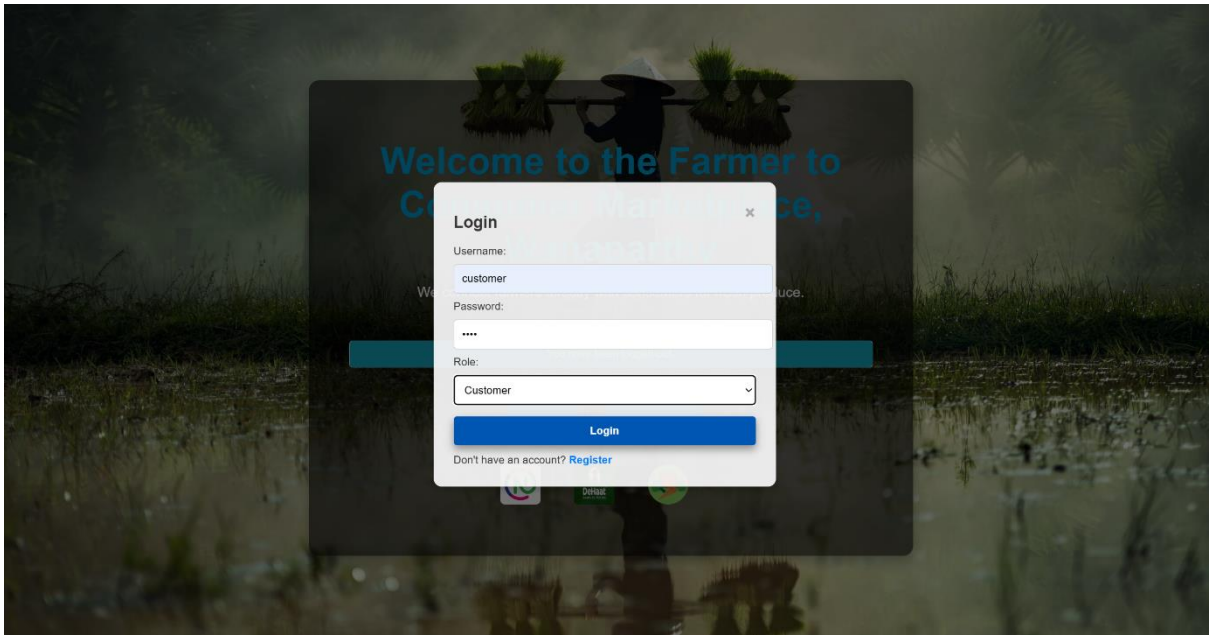
After entering the details click on update profile.

Logout page:



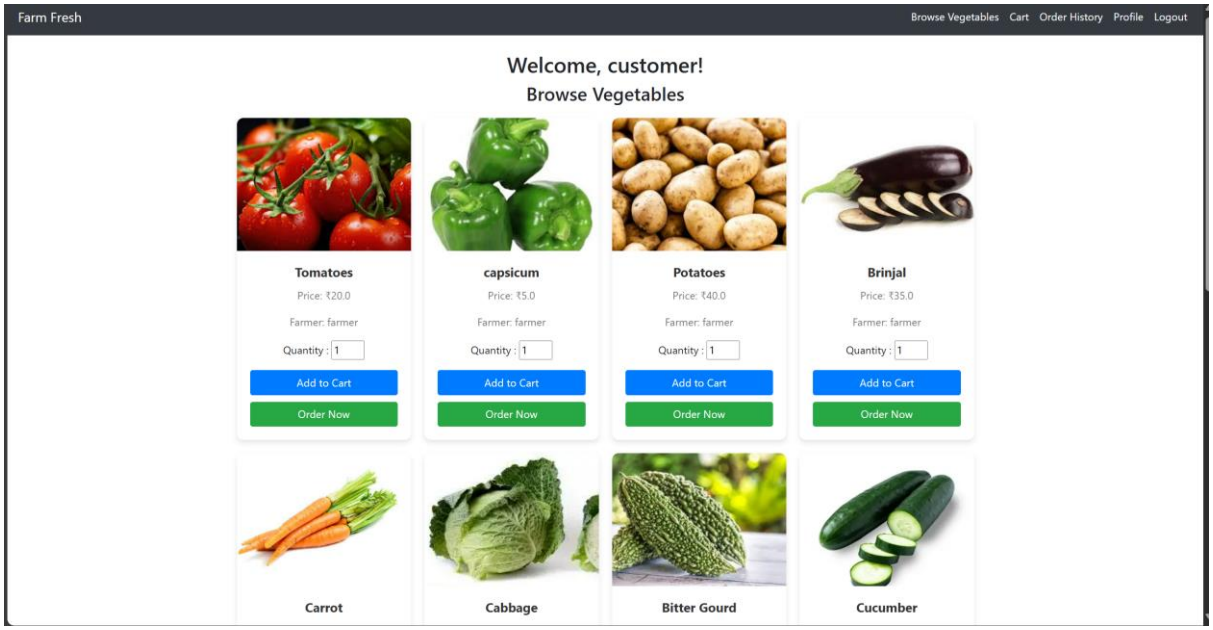
Logging out from the farmer page.

Customer home page:



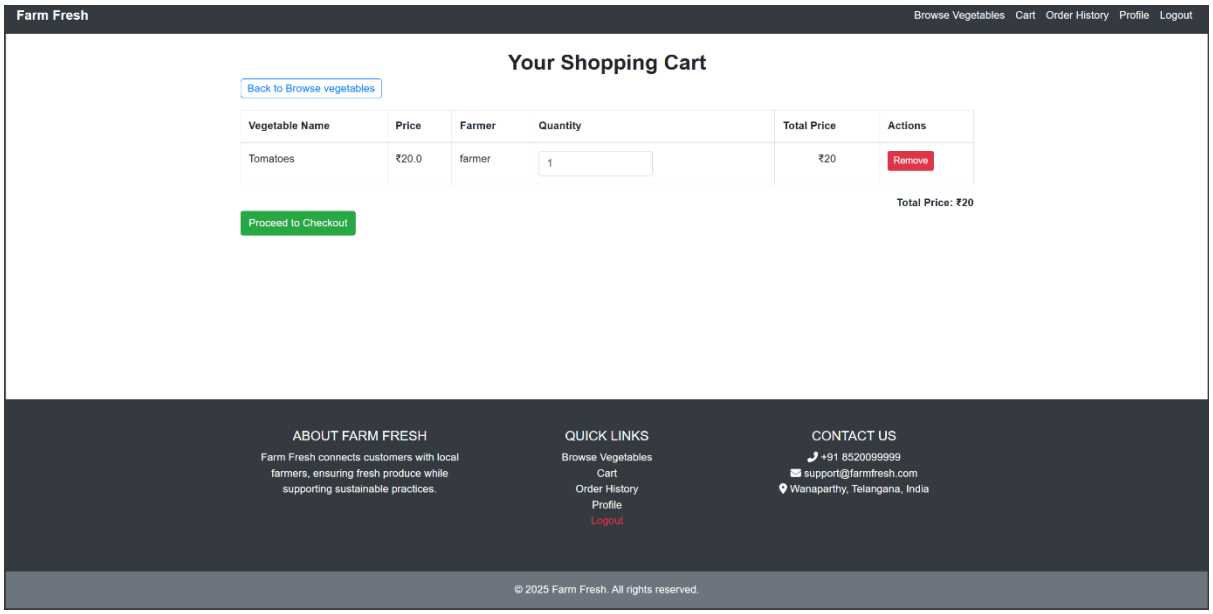
This the process as we did in the farmer page but, we have to select our role as customer and enter our details and click on button.it opens customer page.

Browsing vegetables:



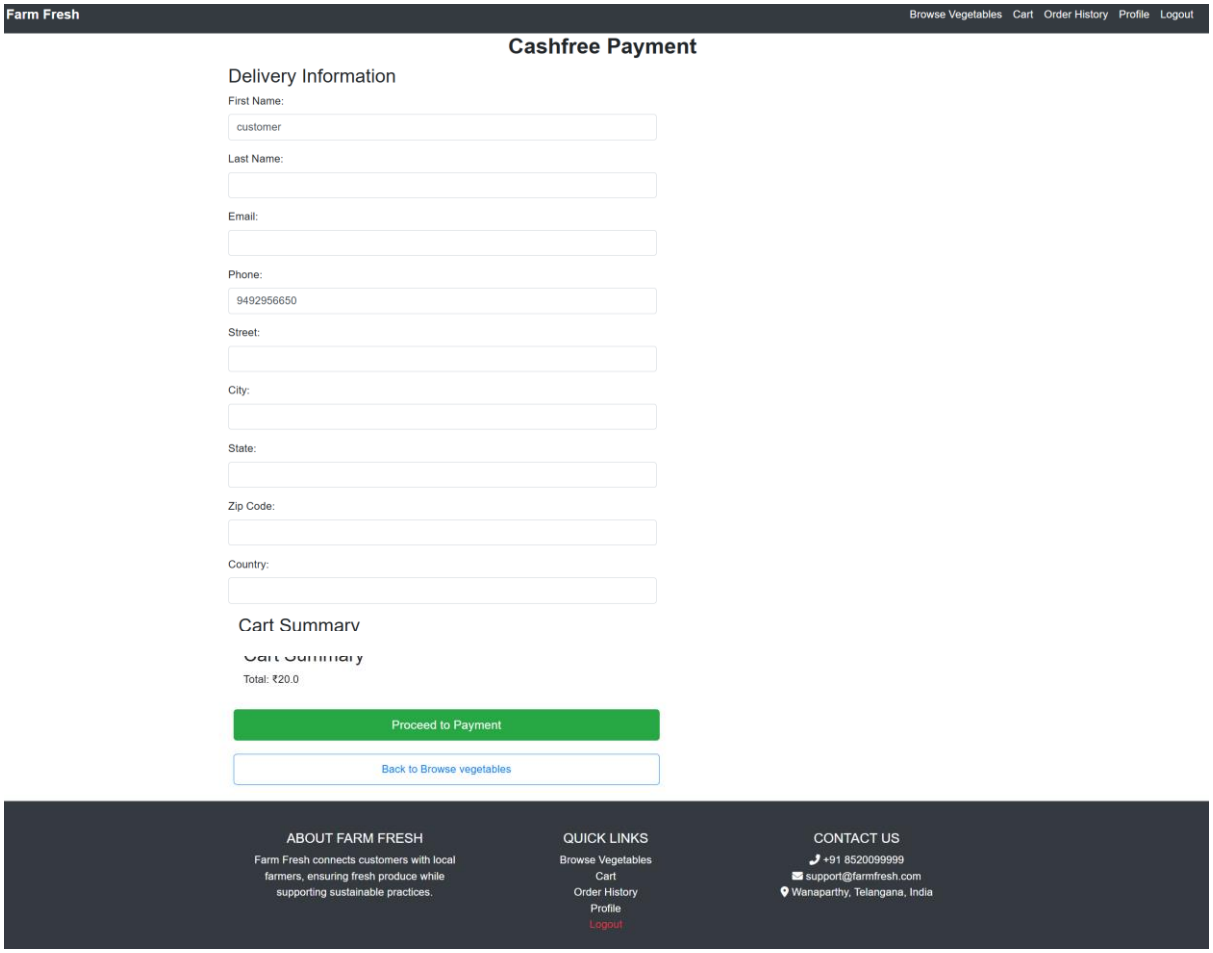
In this page we can see different vegetables from different farmers and can add vegetables to cart or order the vegetables directly .we can specify the quantity of vegetables we required and click on add to cart or order button.

Cart:

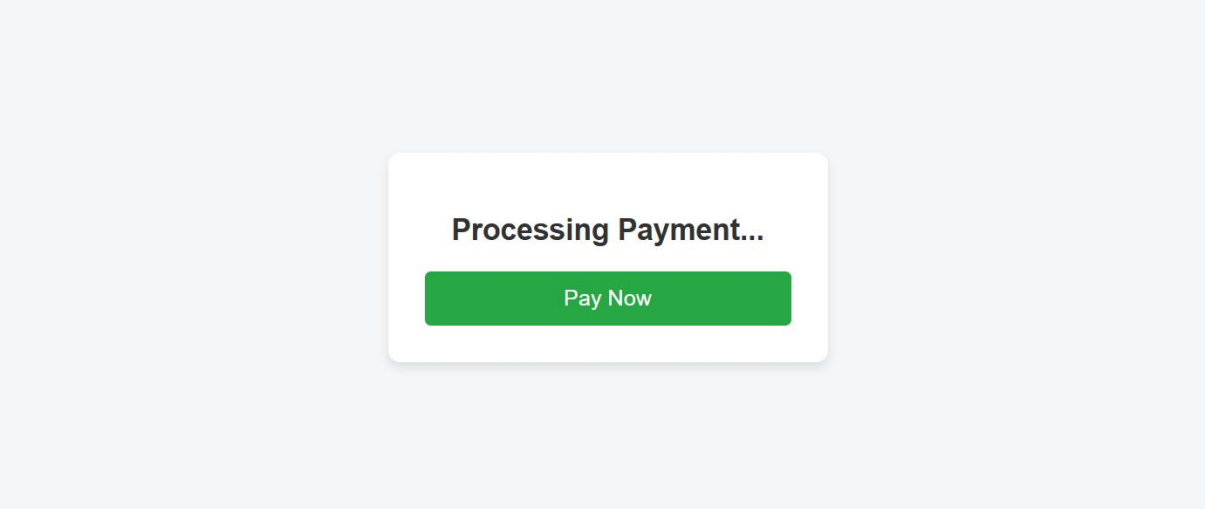


In this page we can update the amount of quantity we required and can remove the items we added to cart by clicking on remove button.by clicking on **proceed to checkout** we can go to payment section.

Payment method:

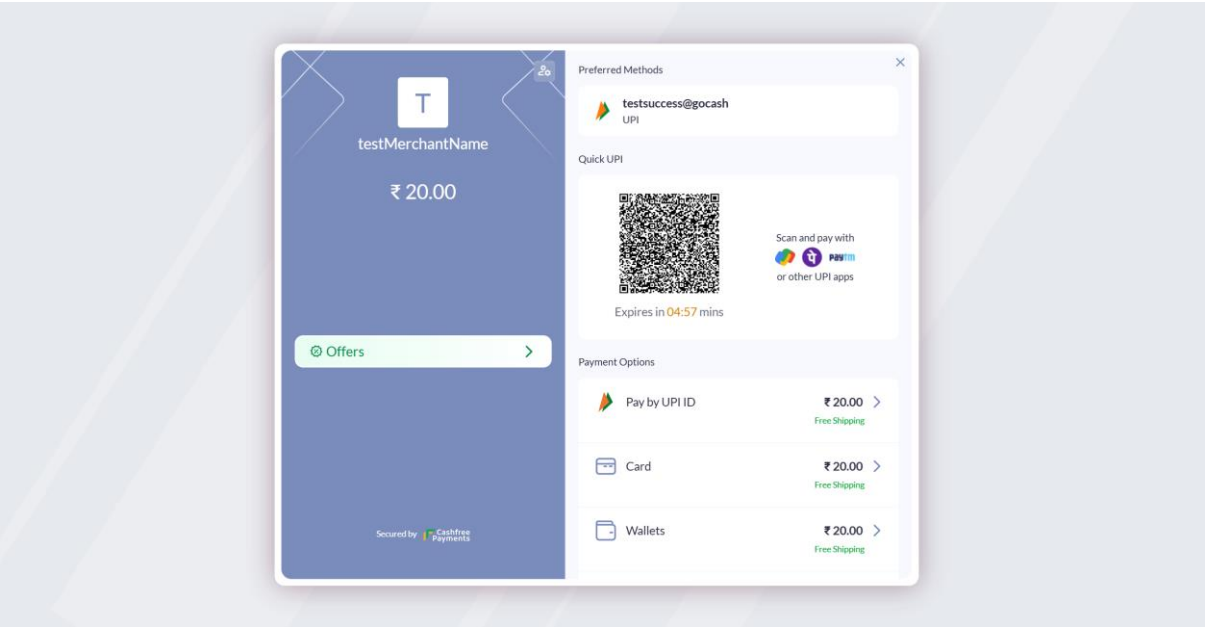


In payment method we have to enter our details like name ,email ,phone ,address after filling the details click on proceed to payment it redirects you to

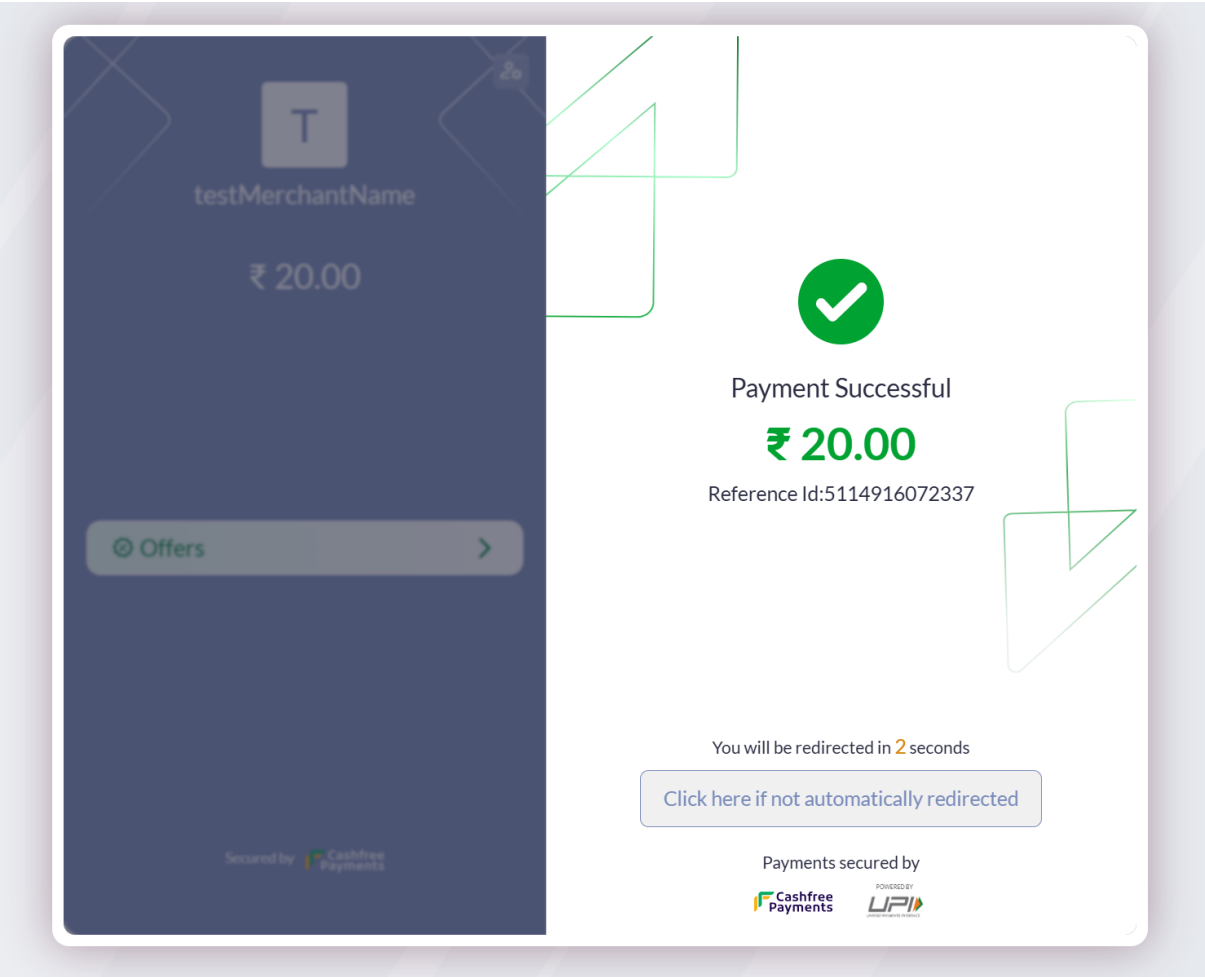


By clicking on pay now it shows the different methods to pay, you can pay by scanning QR code or by UPI id and others.

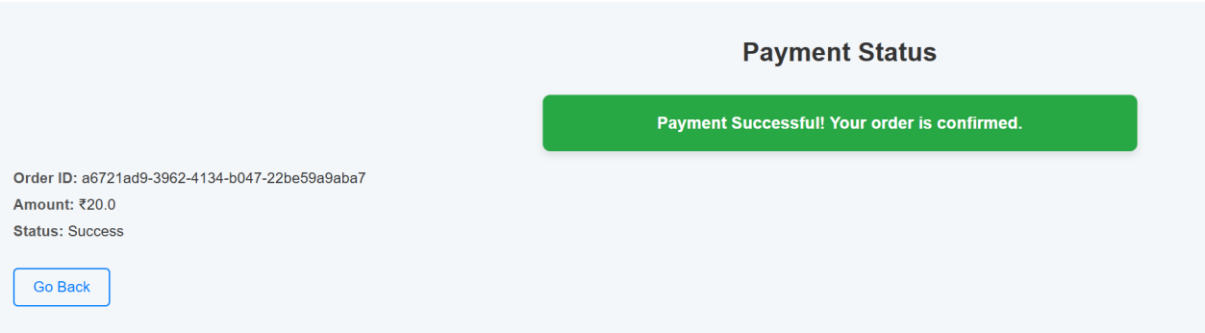
Select the method to pay



After payment it redirects to payment successful page



This page confirms the payment status and clicking on go back it redirects to customer home page.



9.Future Enhancements

I'll add a detailed explanation under the **Future Enhancements** section in your API documentation. Here's how I'll structure it:

1. Automate Payment Verification with Webhooks

Currently, the system relies on manual or periodic checks to verify payment status. By integrating **webhooks**, the verification process will be automated, allowing real-time updates.

Implementation Plan:

- **Webhook Setup:** Configure a webhook endpoint that listens for payment status updates from the payment gateway (e.g., Cashfree).
- **Event Handling:** When a payment is completed, the webhook will trigger and send transaction details to the system.
- **Security Measures:** Validate the webhook data by checking the signature, transaction ID, and payment status to prevent fraud.
- **Automatic Order Status Update:** Upon receiving a successful payment notification, the system will automatically mark the order as "Paid" in the database.
- **Error Handling & Logging:** Implement error handling and retry mechanisms to ensure webhook requests are processed correctly.

Benefits:

- Eliminates manual payment verification.
- Reduces delay in order processing.
- Provides real-time updates to customers and farmers.

2. Implement Email Notifications for Successful Transactions

To enhance customer experience and improve transaction transparency, the system will automatically send email notifications for successful payments.

Implementation Plan:

- **SMTP Integration:** Use an email service provider like **SendGrid, AWS SES, or Gmail SMTP** to send transactional emails.
- **Triggering Emails:** Upon successful payment (verified via webhooks), an email will be sent to the customer with order details, payment confirmation, and invoice (if applicable).
- **Personalized Email Content:**
    - Subject: **Payment Successful – Order #12345**
    - Body: A thank-you message with order details, transaction ID, and expected delivery time.
- **Error Handling:** Log undelivered emails and retry sending in case of failures.

Benefits:

- Provides instant confirmation to users.
- Reduces payment-related queries from customers.
- Enhances trust and transparency.

3. Improve UI with a Better Dashboard Interface

A more intuitive and user-friendly **dashboard interface** will be designed for farmers to manage their inventory, track orders, and monitor sales efficiently.

Enhancements:

- **Modern UI Design:** Improve the dashboard layout with better navigation and a visually appealing design using **React.js, Tailwind CSS, or Bootstrap**.
- **Real-Time Order Updates:** Integrate WebSockets to show live order updates instead of requiring manual page refresh.
- **Graphs & Analytics:** Add **charts and graphs** to display total sales, order trends, and payment summaries.
- **Inventory Management:**
    - Allow bulk updates for vegetable prices.
    - Display stock alerts when inventory is low.
- **Mobile Responsiveness:** Ensure a smooth user experience on both desktop and mobile devices.

Benefits:

- Enhances user experience for farmers.
- Provides better insights into sales and inventory.
- Reduces manual work through automation.

10.Conclsion

The **Farm Fresh Management System** is a comprehensive platform designed to facilitate seamless interactions between farmers and customers, ensuring the efficient sale and purchase of fresh vegetables. The system incorporates a well-defined user flow, robust authentication mechanisms, and a structured database to support essential functionalities such as product listing, order placement, inventory management, and payment processing.

One of the significant achievements of this project is the **role-based access system**, which categorizes users into two primary roles: **farmers and customers**. Farmers can list their products, update stock availability, and manage orders, while customers can browse available vegetables, add them to their cart, and proceed with purchases. This separation of functionalities ensures clarity and efficiency in operations.

The **database design** effectively organizes data into multiple collections, including farmers, customers, vegetables, carts, history, and records. This structure allows seamless data retrieval and updates, ensuring real-time inventory management and accurate order tracking. The **cart system** enables customers to modify their selections before proceeding to checkout, while the **order history feature** provides transparency by allowing users to track past purchases, order status, and payment details.

To enhance security, the system employs **hashed password storage** and session management to protect user data. Additionally, **data validation measures** are in place to ensure input correctness, such as enforcing valid phone numbers, unique usernames, and secure authentication processes.

The **payment gateway integration** facilitates secure transactions, allowing customers to complete their purchases without third-party interference. Furthermore, the system records delivery details, ensuring efficient order fulfillment and tracking.

From a usability perspective, the **intuitive interface** and **structured workflow** contribute to a smooth user experience. The **real-time inventory updates** help prevent stock mismanagement, ensuring that customers only see available products. Farmers can also update vegetable details, including prices, stock levels, and images, making it easier to maintain accurate listings.

Overall, the **Farm Fresh Management System** streamlines direct interactions between farmers and customers, reducing supply chain inefficiencies while promoting fresh and organic produce availability. In the future, enhancements such as **AI-driven demand prediction, automated notifications, and improved UI/UX elements** could further optimize the system, increasing its efficiency and user satisfaction. This project successfully demonstrates the potential of technology in improving agricultural commerce, fostering a more direct and transparent connection between food producers and consumers.